Calculabilitate

- Seminar 1 -

October 9, 2022

Contents

1 Motivație **1**

2 Tipuri de probleme **1**

3 Calculabilitate **2**
3.1 Cât de puternic este un limbaj de programare? 2

4 Tipuri de mulțimi **3**

5 Exerciții Rezolvate **4**

6 Exerciții **7**

7 Observații **7**

1 Motivație

Ne propunem sa identificăm acele probleme care pot fi rezolvate mecanic. Vom descoperi că există un număr foarte mare de probleme practice pentru care nu putem construi o soluție care să meargă pentru **orice** intrare validă. De exemplu:

- Dacă modificăm un program, nu putem verifica automat dacă noul program este echivalent computațional (pentru aceleași intrări, produce același rezultat) cu programul inițial.
- Dat fiind un program oarecare și o intrare validă, nu putem estima automat proprietăți precum timpul de execuție, memoria consumată sau de câte ori sunt citite informații de pe disc.
- Nu putem verifica automat dacă un program oarecare este vulnerabil (e.g. un utilizator neautorizat poate accesa informații confidențiale). Analog, nu putem verifica automat dacă un program este un virus sau nu.

2 Tipuri de probleme

Printr-o problemă înțelegem o mulțime de întrebări, referitoare la proprietățile unor entități, care acceptă răspunsuri precise, finite și a căror corectitudine poate fi riguros demonstrată. Există mai multe tipuri de probleme, dintre care menționăm:

- **Probleme de decizie**: Verifică o proprietate a datelor de intrare (ex: "Este numărul X prim?"). Soluțiile sunt de tip boolean: "DA"/"NU", "Adevarat/Fals", etc.
- **Probleme de optimizare**: Selectează soluția/soluțiile care optimizează o funcție de cost (ex: "Care este cel mai mare divizor prim al numărului X?"). În multe cazuri, problemele de optimizare pot fi reformulate ca probleme de decizie.

3 Calculabilitate

Fie funcția $f_P : I \to O$ care leagă fiecare intrare validă pentru problema Q de răspunsul corect pentru intrarea respectivă.

Definitia 3.1. Spunem că f_P este **efectiv calculabilă** (eng. computable) sau **recursivă** dacă există o metodă efectivă de rezolvare a problemei Q. [1, 2]

Definitia 3.2. O **metoda efectivă** (eng. effective method) este o mulțime finită de instrucțiuni, a căror execuție se termină în timp finit și implică o cantitate finită de date pentru a obține un rezultat, mereu același pentru date identice.

În particular, dacă pentru o problemă de decizie există o metodă efectivă de rezolvare vom spune că este **decidabilă** (eng. decidable). De exemplu, verificarea dacă un număr natural este prim este o problemă decidabilă, deoarece putem defini următorul program:

Listing 1: Verificare dacă un număr este prim in pseudo-cod

```
def checkPrime(x)

    if (x <= 1)
        return False

    for (d = 2; d * d <= x; d++)
        if (x % d == 0)
            return False

    return True
```

Acest program poate fi considerat o metodă efectivă fiindcă se termină pe orice intrare și returnează mereu răspunsul corect.

3.1 Cât de puternic este un limbaj de programare?

Un algoritm poate fi scris în cuvinte (dacă descrierea este riguroasă și lipsită de ambiguitate), în pseudocod, într-un limbaj de programare (C, Java, Python), într-un sistem formal (calcul lambda, logică combinațională, formule aritmetice), sau într-un alt limbaj caracteristic unui sistem care permite manipularea de date (automate pe stări, automate pushdown).

Dar sunt aceste medii de programare echivalente computațional? Este posibil ca un anumit limbaj să nu permită scrierea unui algoritm, respectiv să nu poată fi folosit pentru rezolvarea unei anumite probleme? Avem nevoie de un etalon comun pentru a le putea compara. În acest scop vom folosi **mașina Turing**.

Imaginați-vă o mașină care funcționează cu o bandă infinită de simboluri, un set finit de stări interne și un cap de control așezat pe bandă, capabil sa scaneze simbolurile de sub el. Inițial, pe bandă este scrisă doar intrarea, restul de slot-uri din bandă având simbolul nul. Capul este așezat pe primul simbol al intrării, iar starea internă este o anumită stare inițială. Fiecare instrucțiune a mașinii este de forma: "Când mașina este în starea internă q, și capul de

control se află peste simbolul X, suprascrie simbolul X cu Y, intră în starea internă w și mută capul la stânga/la dreapta". Când mașina ajunge într-o combinație simbol/stare pentru care nu are instrucțiune, se blochează. Mașina se oprește și returnează *Adevărat* sau *Fals* când și dacă ajunge într-una din cele două stări aferente fiecărui răspuns.

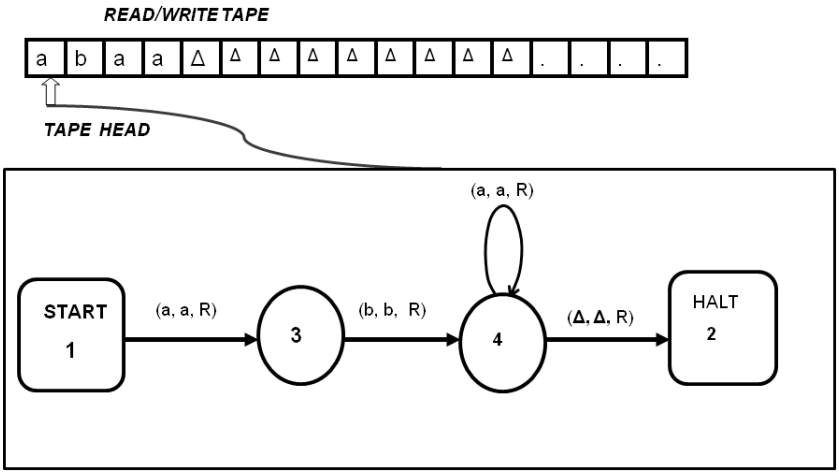


Figure 1: Exemplu Mașină Turing[1]
Pe fiecare muchie avem (*simbol curent*, *simbol înlocuitor*, *direcția de mutat capul*)

Definitia 3.3. O funcție este **calculabilă în sens Turing** (eng. Turing computable) daca există o mașină Turing care se termină în timp finit și returnează răspunsul corect pentru orice intrare validă.

Sună cam primitiv, nu? Pentru a specifica o mașina Turing care verifică dacă un număr este prim am avea nevoie de sute de stări interne, respectiv să executăm mii de instrucțiuni pentru a realiza verificarea propriu-zisă. Din acest motiv, nimeni nu folosește în practică mașinile Turing pentru a descrie algoritmi. Totuși, mașina Turing are o importanță teoretică foarte mare:

Teza Church Turing. Orice funcție este **efectiv calculabilă** dacă și numai dacă este **calculabilă în sens Turing**.

Teza enunța că **orice** problemă care poate fi rezolvată printr-o metodă efectivă poate fi rezolvată și de o mașină Turing.

Definitia 3.4. Spunem că un mediu de programare este **Turing complete** dacă poate fi folosit pentru a simula orice mașina Turing.

Practic, mediile de programare Turing complete sunt echivalente computațional, ele putând să rezolve aceleași probleme, respectiv toate problemele care pot fi rezolvate.

4 Tipuri de mulțimi

În acest laborator vom considera că mulțimile intrărilor valide ale programelor sunt ***numărabile***, adică exista o funcție bijectiva intre acestea și mulțimea numerelor naturale. O asemenea funcție bijectivă definește o numerotare a mulțimii/o ordine a elementelor ei. Un exemplu de mulțime numărabilă este mulțimea șirurilor de caractere peste un alfabet finit, pentru că acestea pot fi ordonate(a se vedea pentru un exemplu observația 1 din secțiunea 7).

[1]Animație MT: http://www.asethome.org/mathfoundations/tmd/

- Fie funcția $f_P : I \to Bool$ asociată problemei de decizie Q.
- (Reminder) Dacă există un program care pentru orice intrare $x \in I$ calculează în timp finit răspunsul pentru problema Q, atunci Q este **decidabilă**.
- Fie $A \subseteq I$ mulțimea de intrări pentru care răspunsul corect la problemă este "Da"/"Adevărat".

Definitia 4.1. O mulțime A este **recursivă** dacă și numai dacă există un program P care răspunde în timp finit dacă un element aparține sau nu mulțimii A.

$$P(x) = \begin{cases} 1 & \text{pentru } x \in A \\ 0 & \text{pentru } x \notin A \end{cases}$$

Definitia 4.2. Spunem că o mulțime A este **recursiv-numărabilă** dacă (alternativ):

- Există un program care calculează răspunsul în timp finit pentru un element $x \in I$ dacă $x \in A$ (dar poate să cicleze la infinit pentru $x \notin A$).

$$P(x) = \begin{cases} 1 & \text{pentru } x \in A \\ \perp & \text{pentru } x \notin A \end{cases}$$

- Există un program G care la fiecare apel generează succesiv un element din A. Cu alte cuvinte, orice element $x \in A$ va fi returnat după un număr finit de apeluri ale funcției G.

5 Exerciții Rezolvate

1. Demonstrați că următoarele proprietăți sunt echivalente pentru o mulțime A (care în cazul în care sunt îndeplinite este recursiv enumerabilă):

   - Există un program P astfel încât:

   $$P(x) = \begin{cases} 1 & \text{pentru } x \in A \\ \perp & \text{pentru } x \notin A \end{cases}$$

   - Există un program G care la fiecare apel generează un element din A și garantează că va returna oricare element din A după un număr finit de apeluri.

   Presupunem ca mulțimea intrărilor pentru programul P este mulțimea numerelor naturale.

   Rezolvare:
   Începem cu demonstrația $P \to G$ (folosind P putem obține G).

   Programul $G()$ ar trebui să întoarcă la fiecare apel un element nou din mulțimea A. Pentru asta, vom folosi o variabilă statică A care va memora elementele întoarse deja și își va păstra starea între apelurile funcției.

```
G(){
    static A = {}
    for (t = 0; ; t++)
        for (x = 0; x <= t; x++)
            if (x ∉ A) {
```

```
                    ruleaza P(x) pentru t pasi
                    if (P(x) s-a oprit) {
                        A = A ∪ {x}
                        return x
                    }
                }
        }
```

Ca să verificăm, trebuie să ne punem următoarele întrebări despre funcția $G()$:

- Este posibil ca ea să returneze o valoare care nu este membru a mulțimii A?
  Nu, deoarece $G()$ nu returnează decât valorile de input pentru care programul P s-a oprit, iar prin definiție programul P nu se oprește decât pentru valori membre ale lui A.
- Este posibil ca un element al lui A să nu fie returnat niciodată de $G()$?
  Nu, pentru orice membru x al lui A, programul P se va opri după un număr finit de pași, e. După un număr finit de apeluri succesive ale lui $G()$, următorul apel va ajunge la un pas $t \geq max(e, x)$ în care x va fi adăugat la soluție și va fi întors de $G()$.

Acum mai trebuie să demonstrăm dependența $G \rightarrow P$, care este mai simplă:
Presupunem că există un generator G și îl definim pe P astfel:

```
int P(int x) {
    while (G() != x);
    return 1;
}
```

Din moment ce $G()$ garantează că va returna oricare intrare acceptată după un număr finit de apeluri, avem garanția și că P se va termina dacă și numai dacă $x \in A$. Deci P este corect definit.

Deducțiile în ambele sensuri au fost demonstrate deci echivalența este adevărată.

2. Fie două mulțimi, A și $B = N \backslash A$, ambele recursiv-enumerabile. Demonstrați că ambele sunt recursive.

Rezolvare:
Dacă cele două mulțimi sunt recursiv-enumerabile inseamnă că fiecare are câte o funcție generator, fie GA și GB. Cu ele putem defini următorul program:

```
int P(int x) {
    int sol;
    while (true) {
        if (GA() == x) {
            sol = 1;
            break;
        }
        if (GB() == x) {
            sol = 0;
```

```
            break;
        }
    }
    return sol;
}
```

Din moment ce A și B sunt complementare știm că orice $x \in N$ aparține fie lui A fie lui B. Fiecare dintre funcțiile generator garantează să returneze orice membru al său după un număr finit de apeluri deci P este garantat să se termine și să returneze 1 pentru $x \in A$ și 0 pentru $x \in B$.
Acest program demonstrează recursivitatea ambelor mulțimi. Pentru a respecta definiția și pentru mulțimea B nu trebuie decât să negăm rezultatul lui P.

3. Fie mulțimile A, B, C astfel încât: A este recursivă, $A \cup B$ nu este nici măcar recursiv-enumerabilă, iar $B \oplus C$ este recursivă.
Ce se poate spune despre B și C?

Rezolvare:
În continuare ne vom referi la mulțimea mulțimilor recursive ca R, mulțimea mulțimilor recursiv enumerabile ca RE și mulțimea mulțimilor nici măcar recursiv enumerabile ca NRE.
Din moment ce $A \in R$ și $A \cup B \in NRE$ intuim că $B \in NRE$. Vom demonstra acest lucru printr-o contradicție. Prespunem că B este recursiv-enumerabilă, deci există un program PB care se oprește și returnează 1 pentru toți membrii lui B. Știm că A este recursivă deci există un program PA care se termină pe orice input și returnează corect aparenența la A.
Vom defini următorul program $PAuB$:

```
int PAuB(int x) {
    if PA(x) return 1;
    else return PB(x);
}
```

Acest program se termină și returnează 1 pentru orice $x \in A \cup B$. Asta inseamnă că $A \cup B \in RE$ dar $A \cup B \in NRE$. Contradicție.
De aici rezultă că B nu este nici măcar recursiv-enumerabilă, $B \in NRE$.

Fiindcă $B \oplus C$ este recursivă iar B nu este nici măcar recursiv-enumerabilă putem deduce faptul că C este nerecursivă. Facem acest lucru din nou prin reducere la absurd: Presupunem că C este recursivă, deci există un program PC care se oprește și returnează apartenența intrării la mulțimea C.
Pe programul care rezolvă $B \oplus C$ îl vom numi $PBoC$. Atfel, putem să definim PB:

```
int PB(int x) {
    if PBoC(x) return not PC(x);
    else return PC(x);
}
```

Acest program efectuează în esență: $(B \oplus C) \oplus C = B$. Din moment ce se termină și calculează corect apartenența la B, rezultă că B este recursivă, dar deja am stabilit că B nu este nici măcar recursiv-enumerabilă. Contradicție.
Rezultă că C este nerecursivă.

6 Exerciții

1. Fie mulțimile A, B, C disjuncte ($A \cap B = \emptyset$, $B \cap C = \emptyset$, $A \cap C = \emptyset$), și complementare ($A \cup B \cup C = \mathbb{N}$). Demonstrați că dacă A, B și C sunt recursiv-enumerabile, atunci acestea sunt și recursive.

2. Fie mulțimea A recursivă și mulțimea B nerecursivă dar recursiv-enumerabilă. Este mulțimea $A \cup B$ recursiv-enumerabilă?

3. *Bonus: De la exercițiul anterior, demonstrați prin exemple că mulțimea $A \cup B$ poate fi și recursivă și nerecursivă.

4. Mulțimea A este recursiv-enumerabilă dar nerecursivă. Ce putem spune despre mulțimea $B = N \setminus A$?

5. Fie mulțimea A nici măcar recursiv-enumerabilă, și mulțimea $B \subset A$ astfel încât $A \setminus B$ este recursivă. Ce putem spune despre B?

6. *Bonus: Fie mulțimea A recursiv enumerabilă, și mulțimea $B = \{x + 1 | x \in A\}$. Știind că $A \cap B = \emptyset$, demonstrați că dacă $A \cup B$ recursivă atunci și A și B sunt recursive.

7. *Bonus: Fie mulțimea $A \subset N$ care nu are nicio pereche de elemente consecutive și mulțimea $B = \{x + 1 | x \in A\}$. Demonstrați că dacă $A \cup B$ recursivă atunci și A și B sunt recursive.

7 Observații

1. Mulțimea șirurile unui alfabet finit este recursiv-enumerabilă din moment ce putem ordona șirurile după mărime crescător și lexicografic. Spre exemplu mulțimea șirurilor cu caracter din alfabetul a, b poate fi ordonată ["", "a", "b", "aa", "ab", "ba", "bb", "aaa"...].
Pentru orice mulțime recursiv enumerabilă există o metodă efectivă de a trece de la elemente din ea la indicele lor, aferent ordonării stabilite, și invers, de la indice la element. Deci practic în rezolvări când ne folosim de intrare ca de un număr natural, noi facem implicit conversia spre indicele ei. Când avem nevoie de intrarea problemei, facem implicit conversia înapoi.

2. Teza Church-Turing nu este demonstrată decât pe diverse formalizări ale ideii de funcție efectiv calculabilă, acesta în sine fiind un termen informal, care nu poate fi introdus într-o demonstrație formală. Totuși, accepțiunea comunității este că teza este adevărată.

References

[1] Cristian A. Giumale. *Introducere în Analiza Algoritmilor*. Polirom, 2004.

[2] Neil Immerman. Computability and complexity. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, winter 2018 edition, 2018.

Metode de a demonstra (ne)decidabilitatea unor probleme

19 octombrie 2022

1 De ce studiem aceste metode?

În capitolul anterior am aflat că nu toate problemele sunt decidabile. Există probleme *decidabile* (recursive), *semidecidabile* (recursive enumerabile, dar nu recursive - cărora le putem identifica instanțele-*da*, dar nu le putem identifica instanțele-*nu*), și probleme care nu sunt nici măcar semi-decidabile (nu sunt recursive enumerabile). În capitolul curent:

- dăm exemple de probleme semidecidabile, și înțelegem natura lor: sunt probleme de căutare într-un spațiu infinit, care, atunci când lucrul căutat nu există, nu au cum să știe că ar trebui să înceteze căutarea;
- introducem mecanismul de reducere prin mapare (a intrărilor unei probleme în intrări ale altei probleme), ilustrat prin reducerea Turing; această tehnică ne permite să demonstrăm că anumite probleme nu sunt recursive sau recursiv enumerabile; reducerile joacă un rol important atât în teoria decidabilității (capitolul curent), cât și în teoria complexității (vom vedea mai târziu);
- realizăm o serie de reduceri Turing, un antrenament pentru a recunoaște șabloanele comune unor probleme aparent diferite; abilitatea de a recunoaște astfel de șabloane în contexte neevidente se traduce în abilitatea de a recunoaște într-o problemă nouă o problemă pe care am mai întâlnit-o și rezolvat-o, adică în mai multă eficiență în rezolvarea problemelor;

2 Reducerea Turing

2.1 Definiție

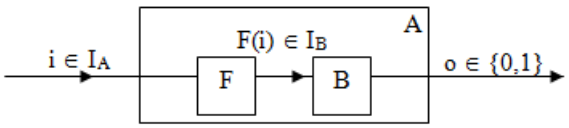


Figura 1: Reducerea Turing a unei probleme A la o problemă B

Spunem că problema A se reduce Turing la problema B ($A \leq_T B$) dacă și numai dacă există o funcție (implementabilă) F care transformă orice intrare i a problemei A într-o intrare $F(i)$ pentru problema B, astfel încât $A(i) = 1 \Leftrightarrow B(F(i)) = 1$. Deci pentru ca transformarea F să fie o reducere Turing valabilă, trebuie să existe un program sau o mașină Turing care într-un număr finit de pași să o efectueze. Altfel spus, o demonstrație matematică (neconstructivă) ca o asemenea funcție există nu este îndeajuns.

Demonstrația că $A \leq_T B$ se realizează în 3 pași: găsirea funcției de transformare F, demonstrarea implicației de la stânga la dreapta, și demonstrarea implicației de la dreapta la stânga.

Intuiția informală este că A este mai ușoară (sau la fel de ușoară) ca B, în sensul că, dacă știm să rezolvăm B, automat știm și să rezolvăm A. $\leq_T$ se comportă ca o relație de ordine între dificultatea problemelor, din punct de vedere al decidabilității.

Observația 1: Reducerea Turing nu este o relație simetrică, deci dacă A se reduce la B nu

înseamnă că și B se reduce la A. Fiecărei instanțe i (aleasă aleator) din A îi corespunde o instanță (construită convenabil, în funcție de i) din B. Întregului set de intrări pentru A i se asociază doar un subset din intrările lui B.

Observația 2: Echivalența este o dublă implicație, care trebuie demonstrată în ambele sensuri (întâi $A(i) = 1 \Rightarrow B(F(i)) = 1$, apoi $B(F(i)) = 1 \Rightarrow A(i) = 1$; demonstrând doar prima parte, arătăm că B nu transformă instanțele-*da* ale lui A în instanțe-*nu*, dar nu garantăm că B nu inventează instanțe-*da* noi).

Observația 3: Ca orice relație de ordine, $\leq_T$ este reflexivă, antisimetrică ($A \leq_T B$ și $B \leq_T A \Leftrightarrow A \equiv_T B$), și tranzitivă. Aceste proprietăți sunt ușor de demonstrat și sunt lăsate ca exercițiu.

2.2 Consecințe

1. $A \leq_T B$ și $B \in \mathcal{R} \Rightarrow A \in \mathcal{R}$ (rezultă ușor din definiția reducerii Turing)
2. $A \leq_T B$ și $B \in \mathcal{RE} \Rightarrow A \in \mathcal{RE}$ (rezultă ușor din definiția reducerii Turing)
3. $A \leq_T B$ și $A \notin \mathcal{R} \Rightarrow B \notin \mathcal{R}$ (altfel ar deveni și A recursivă, conform 1.)
4. $A \leq_T B$ și $A \notin \mathcal{RE} \Rightarrow B \notin \mathcal{RE}$ (altfel ar deveni și A recursiv enumerabilă, conform 2.)

Aceste consecințe se folosesc pentru a demonstra (ne)decidabilitatea unor probleme, folosind reduceri (de) la probleme a căror (ne)decidabilitate este cunoscută.

3 Demonstrarea nedecidabilității unor probleme

3.1 Exemple de probleme nedecidabile

- Problema opririi (se oprește un program arbitrar P pe un input arbitrar w?);
- Determinarea validității unei propoziții arbitrare în logica cu predicate de ordinul întâi;
- Rezolvarea unei ecuații diofantice arbitrare;
- Post Correspondence Problem.

3.2 Post Correspondence Problem (PCP)

3.2.1 Enunț și exemple

Dându-se 2 liste W și X a câte n cuvinte fiecare ($W = [w_1, w_2, ..., w_n]$ și $X = [x_1, x_2, ..., x_n]$), există o secvență nevidă de k întregi $i_1, i_2, ..., i_k$ astfel încât $w_{i_1} w_{i_2} ... w_{i_k} = x_{i_1} x_{i_2} ... x_{i_k}$?(unde $w_{i_1} w_{i_2} ... w_{i_k}$ reprezintă concatenarea cuvintelor $w_{i_1}, w_{i_2}, ..., w_{i_k}$, iar secvența nu trebuie să fie de k întregi diferiți între ei, ci același indice poate fi folosit de oricâte ori)

Intuitiv, pentru fiecare k (1, 2, 3, ...), putem încerca toate combinațiile de indici. Dacă există o soluție, o vom găsi mai devreme sau mai târziu; dacă însă nu există, nu vom avea niciodată garanția că nu vom găsi o soluție pentru un k mai mare decât cel curent.

Putem să ne imaginăm perechile cu același indice ca pe niște dominouri pe care trebuie să le așezăm unul lângă altul până când partea de sus arată ca partea de jos (cu precizarea că putem folosi același domino de oricâte ori). De exemplu, pentru intrarea $W = [a, bb, a], X = [aa, b, bb]$, rezultă dominourile de mai jos, iar o soluție posibilă este *(1, 3, 2, 2)*, care produce, și "sus" și "jos", cuvântul *aabbbb*.

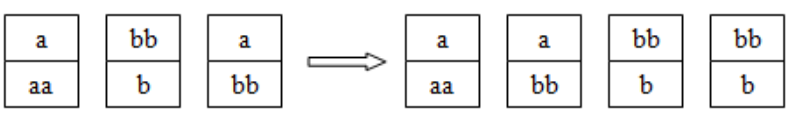


Figura 2: O pereche intrare-ieșire pentru PCP

Exerciții: Găsiți (sau argumentați că nu există) soluții pentru următoarele instanțe PCP:

1. $W = [ab, baa, aba], X = [aba, aa, baa]$
2. $W = [bb, ab, c], X = [b, ba, bc]$

3.2.2 Exemplu de reducere Turing: MPCP $\leq_T$ PCP

Prin MPCP (Modified PCP), înțelegem problema PCP cu restricția că soluțiile trebuie să înceapă cu indicele 1 (cu perechea (w_1, x_1)). Demonstrăm MPCP $\leq_T$ PCP.

Pasul 1: pentru o intrare oarecare W, X a lui MPCP, construim o intrare convenabilă W', X' a lui PCP, astfel încât $MPCP(W, X) = 1 \Leftrightarrow PCP(W', X') = 1$.
Truc: Introducem în alfabet două simboluri noi, * și $.

- față de cum arată W în MPCP, în PCP introducem * după fiecare simbol;
- față de cum arată X în MPCP, în PCP introducem * înainte de fiecare simbol (de exemplu, dominoul (ab, bb) devine $(a * b*, *b * b)$);
- corespunzând dominoului cu indice 1 ((w_1, x_1)), adăugăm în plus un domino cu simbolurile * plasate după regula de mai sus, plus un simbol * în fața lui w_1, și numim aceast domino dominoul 0 (de exemplu, dominoul (baa, ab) generează dominoul 0 $(*b * a * a*, *a * b)$);
- adăugăm dominoul final, $(\$, *\$)$.

Astfel ne-am asigurat că orice soluție pentru PCP începe cu dominoul 0 (singurul în care cuvintele încep cu același simbol) și se termină cu dominoul final (singurul în care cuvintele se termină cu același simbol).

Pasul 2: $MPCP(W, X) = 1 \Rightarrow PCP(W', X') = 1$.
Dacă MPCP are o soluție, ea este de forma $(1, i_2, i_3, ..., i_k)$, ceea ce face ca $(0, i_2, i_3, ..., i_k, i_{final})$ să fie soluție pentru PCP.

Pasul 3: $PCP(W', X') = 1 \Rightarrow MPCP(W, X) = 1$.
Dacă PCP are o soluție, ea este de forma $(0, i_2, i_3, ..., i_k, i_{final})$, ceea ce face ca $(1, i_2, i_3, ..., i_k)$ să fie soluție pentru MPCP (practic, se obține ștergând * și $ din soluția pentru PCP).

Exerciții:

1. Este PCP decidabilă pe un alfabet unar?
2. Este PCP decidabilă pe un alfabet binar?

3.3 Probleme înrudite cu problema opririi

3.3.1 Șabloane generale de reducere

Dacă vrem să demonstrăm că o problemă nu este recursivă (decidabilă), vom face o reducere Turing de la o problemă despre care știm că nu este recursivă (cum ar fi problema opririi) la problema noastră. Dacă vrem să demonstrăm că problema nu este nici recursiv enumerabilă, vom face reducerea de la complementul problemei opririi. Aceste șabloane se vor înțelege mai ușor după rezolvarea câtorva exerciții):

- Dacă ne interesează doar proprietăți ale setului de intrări pe care un program P' se termină, vom construi P' ca pe un program care își ignoră inputul și rulează la un moment dat $P(w)$, astfel încât proprietatea căutată să se respecte doar când $P(w)$ se termină.
- Dacă ne interesează un comportament al programului P' (în afara proprietății de a se termina sau nu pe anumite intrări), vom încerca să modificăm P astfel încât să nu manifeste niciodată acel comportament, apoi vom construi P' astfel încât să manifeste comportamentul în cauză doar după ce $P(w)$ se termină.
- Ocazional, inputul lui P' nu este ignorat complet, ci este testat pentru anumite proprietăți ușor de detectat, caz în care programul returnează, sau intră în buclă infinită, sau rulează $P(w)$ returnând doar când acesta returnează.

3.3.2 VID: Are un program arbitrar *P'* proprietatea de a nu se opri pe niciun input?

Demonstrăm că VID nu este recursiv enumerabilă demonstrând că $\overline{PO} \leq_T VID$.

Pasul 1: pentru o intrare oarecare P, w pentru $\overline{PO}$, construim o intrare convenabilă P' a lui VID, astfel încât $\overline{PO}(P, w) = 1 \Leftrightarrow VID(P') = 1$. Ne încadrăm în șablonul 1 de mai sus.

```
P'(x) {
  P(w);
  return 1;
}
```

Conform definiției lui P', acesta se oprește pe orice input ($\Rightarrow VID(P') = 0$) dacă $P(w)$ se oprește, și niciodată ($\Rightarrow VID(P') = 1$) altfel.

Pasul 2: $\overline{PO}(P, w) = 1 \Rightarrow VID(P') = 1$.
Când P ciclează pentru w ($\overline{PO}(P, w) = 1$), P' va cicla și el pentru orice input x ($VID(P') = 1$).

Pasul 3: $VID(P') = 1 \Rightarrow \overline{PO}(P, w) = 1$.
Dacă P' nu se oprește pentru niciun input, înseamnă că $P(w)$ nu se oprește nici el.

Exerciții: Determinați pentru următoarelor probleme dacă ele sunt decidabile (recursive), semi-decidabile (recursiv enumerabile dar nu recursive) sau nu sunt nici semidecidabile (nu sunt recursiv enumerabile):

1. P5: Se termină un program arbitrar P' pe intrarea 5?
2. P5': Se termină un program arbitrar **doar** pe intrarea 5?
3. EXP: Calculează un program arbitrar 2^x pentru orice intrare $x \in \mathbb{N}$?
4. STOP3: Se oprește un program arbitrar P' fix pe 3 intrari?
5. EQP: Se opresc două programe arbitrare, P_1 și P_2, pe aceleași inputuri? (reduceți atât PO, cât și VID la EQP)

4 Concluzii și lecturi recomandate

- Noțiunile de calculabilitate și decidabilitate au un istoric mai complicat decât forma în care ni se prezintă nouă astăzi (vezi www.panspermia.org/egyptians2008.doc);
- Noțiunea abstractă de calculabilitate nu se traduce direct într-o noțiune practică de calculabilitate (vezi teza Church-Turing);
- Cunoașterea nedecidabilității unei probleme ne poate conduce la a căuta cazuri particulare pe care problema să fie decidabilă; adesea problemele nedecidabile au cazuri particulare a căror decidabilitate este încă în dubiu (vezi PCP limitat la 3-6 dominouri);
- Există funcții necalculabile; un exemplu spectaculos este *busy beaver* (vezi http://www.scottaaronson.com/writings/bignumbers.html pentru o discuție interesantă despre puterea paradigmelor în matematică și știința calculatoarelor, cu referire inclusiv la busy beaver);
- Nu există reguli generale pentru identificarea felului în care o problemă se poate transforma într-alta, însă experiența și imaginația ajută foarte mult (vezi http://www.lel.ed.ac.uk/~gpullum/loopsnoop.html pentru un mod distractiv de a prezenta problema opririi).

Notații de complexitate

23 octombrie 2022

1 Ce înseamnă și de ce depinde complexitatea?

Un algoritm performant consumă cât mai puține resurse temporale și spațiale. Cantitatea de resurse folosite depinde de:

- formatul datelor de intrare (ex: în general se consumă mai puțin pentru ordonarea unui vector gata sortat decât a unuia cu elemente aranjate aleator)
- dimensiunea datelor de intrare - notată tipic **n** (ex: se consumă mai mult timp sortând un vector de 1000 de elemente față de unul de 3 elemente)

Măsurarea experimentală a timpului consumat de algoritm introduce dependențe de hardware (mașina pe care se rulează) sau software (limbajul de programare) – lucruri care nu țin de calitatea intrinsecă a algoritmului. Vom prefera o **estimare matematică a numărului de pasi** parcurși de algoritm. În calculul de complexitate ținem cont numai de **operațiile critice** (operațiile care prin natura lor sunt foarte consumatoare sau sunt efectuate de un număr semnificativ de ori; ex: într-un algoritm de sortare o operație critică este comparația de elemente, întrucât se produce de multe ori).

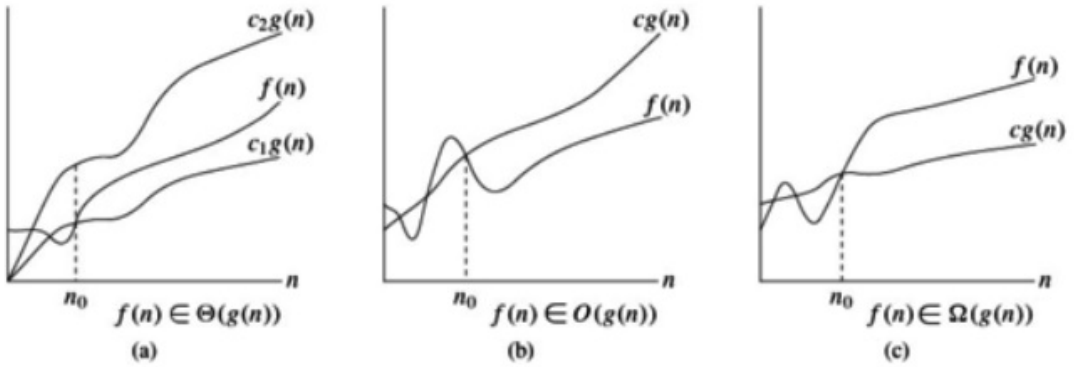
Este dificil de calculat exact un număr de pași, iar ceea ce va interesa este "cam de ce ordin" este funcția de complexitate (în ce **clasă de complexitate**) sau, altfel spus, **cum crește funcția de complexitate** (linear/logaritmic/exponențial). Astfel, se deduce cum se va comporta algoritmul pe dimensiuni mari ale datelor de intrare, acolo unde diferențele se simt cel mai acut. Această idee stă la baza **analizei asimptotice** de complexitate, bazată pe notațiile asimptotice de complexitate.

2 Tipuri de analiză de complexitate

- **cazul cel mai defavorabil** (ce complexitate rezultă pentru cel mai neprietenos input): analiza cea mai frecventă, e ușor de realizat și oferă utilizatorului o garanție: algoritmul nu se va purta niciodată mai rău decât atât;
- **cazul mediu** (la ce complexitate ne putem aștepta în cazul inputurilor aleatoare): o analiză utilă, însă dificil de realizat; necesită cunoașterea unei distribuții statistice a posibilelor inputuri, pentru a pondera pe fiecare cu probabilitatea sa de apariție și a calcula apoi o astfel de medie ponderată a complexităților;
- **cazul cel mai favorabil** (ce complexitate rezultă pentru cel mai prietenos input): analiza cea mai puțin frecventă, utilă cel mult pentru probleme care tind să aibă inputuri favorabile; în plus, este ușor de trișat, prin plasarea unui test pentru un input anume la începutul algoritmului, caz în care se dă direct rezultatul, cu minim de efort;

3 Notațiile asimptotice de complexitate (O, Ω, Θ, o, ω)

Funcțiile de complexitate sunt funcții asimptotic crescătoare de tip $N \to R_+$ (în figură, f(n) reprezintă numărul de pași efectuați de algoritm pentru o intrare de dimensiune n).



1. $O(g(n)) = \{f : N \to R_+ \,|\, \exists\, ct.\, c \in R_+, c > 0, \text{ și } n_0 \in Na.i.\, pt\, \forall n \geq n_0, 0 \leq f(n) \leq cg(n)\}$

   - $f(n) \in O(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c * g(n)$ este o **limită superioară** pentru $f(n)$; algoritmul se va purta mereu mai bine decât această limită;

2. $\Omega(g(n)) = \{f : N \to R_+ \,|\, \exists\, ct.\, c \in R_+, c > 0, \text{ și } n_0 \in Na.i.\, pt\, \forall n \geq n_0, 0 \leq cg(n) \leq f(n)\}$

   - $f(n) \in \Omega(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c * g(n)$ este o **limită inferioară** pentru $f(n)$; algoritmul se va purta mereu mai rău decât această limită;

3. $\Theta(g(n)) = \{f : N \to R_+ \,|\, \exists\, ct.\, c_1, c_2 \in R_+, c_1 > 0, c_2 > 0 \text{ și } n_0 \in Na.i.\, pt\, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

   - $f(n) \in \Theta(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c_1 g(n)$ este o **limită inferioară** pentru $f(n)$, iar $c_2 g(n)$ o **limită superioară**; algoritmul tinde să se comporte "cam ca" $g(n)$.
   - **Pentru analize de complexitate cât mai precise preferăm notația Θ.**

4. $o(g(n)) = \{f : N \to R_+ \,|\, \forall c \in R_+, c > 0, \exists\, n(c) \in Na.i.\, pt\, \forall n \geq n(c), 0 \leq f(n) < cg(n)\}$

   - $f(n) \in o(g(n))$ înseamnă că $g(n)$ crește **strict** mai repede decât $f(n)$

5. $\omega(g(n)) = \{f : N \to R_+ \,|\, \forall c \in R_+, c > 0, \exists\, n(c) \in Na.i.\, pt\, \forall n \geq n(c), 0 \leq cg(n) < f(n)\}$

   - $f(n) \in \omega(g(n))$ înseamnă că $f(n)$ crește **strict** mai repede decât $g(n)$

Recomandare: Pentru continuarea capitolului de complexitate, revizuiți proprietățile logaritmilor, exponențialelor, seriilor aritmetice și geometrice.

Redefinirea notațiilor asimptotice folosind limite de funcții:

- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 \leq L < \infty \Rightarrow f(n) \in O(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 < L < \infty \Rightarrow f(n) \in \Theta(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 < L \leq \infty \Rightarrow f(n) \in \Omega(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = 0 \Rightarrow f(n) \in o(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = \infty \Rightarrow f(n) \in \omega(g(n))$

4 Proprietăți ale notațiilor asimptotice de complexitate:

Tranzitivitatea:

- $f(n) \in O(h(n)) \wedge h(n) \in O(g(n)) \Rightarrow f(n) \in O(g(n))$
- $f(n) \in \Omega(h(n)) \wedge h(n) \in \Omega(g(n)) \Rightarrow f(n) \in \Omega(g(n))$
- $f(n) \in \Theta(h(n)) \wedge h(n) \in \Theta(g(n)) \Rightarrow f(n) \in \Theta(g(n))$
- $f(n) \in o(h(n)) \wedge h(n) \in o(g(n)) \Rightarrow f(n) \in o(g(n))$
- $f(n) \in \omega(h(n)) \wedge h(n) \in \omega(g(n)) \Rightarrow f(n) \in \omega(g(n))$

Reflexivitatea:

- $f(n) \in O(f(n))$
- $f(n) \in \Omega(f(n))$
- $f(n) \in \Theta(f(n))$

Simetria:

- $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$

Antisimetria:

- $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$

Altele:

- $f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$
- $f(n) \in o(g(n)) \Leftrightarrow f(n) \in O(g(n)) \wedge not(f(n) \in \Theta(g(n)))$
- $f(n) \in \omega(g(n)) \Leftrightarrow f(n) \in \Omega(g(n)) \wedge not(f(n) \in \Theta(g(n)))$
- $\Theta(f(n) + g(n)) = \Theta(f(n)) + \Theta(g(n))$
- $\Theta(f(n) * g(n)) = \Theta(f(n)) * \Theta(g(n))$

Exemplu de analiză de complexitate: insertion-sort

```
1:  function INSERTION-SORT(v)                          ▷ Nr. execuții/instrucțiune
2:      for j ← 2 to n do                               ▷ n
3:          elem ← v[j]                                 ▷ n-1
4:          i ← j − 1                                   ▷ n-1
5:          while i > 0 and elem < v[i] do              ▷ S1
6:              v[i + 1] ← v[i]                         ▷ S2
7:              i ← i − 1                               ▷ S2
8:          end while
9:          v[i + 1] ← elem                             ▷ n-1
10:     end for
11: end function
```

1. **Cazul cel mai favorabil:** v deja sortat, nu se intră în while
   - S1 = n-1 (prin fiecare instrucțiune while se trece fix o dată, de n-1 ori)
   - S2 = 0 (nu se ajunge la instrucțiunile din interiorul while-ului) $\Rightarrow T(n) = 5n - 4 \Rightarrow T(n) \in \Theta(n)$
2. **Cazul cel mai defavorabil:** v sortat invers, fiecare while merge până la i=0
   - $S1 = \sum_{j=2}^{n} j = n(n+1)/2–1$
   - $S2 = \sum_{j=2}^{n}(j-1) = n(n-1)/2 \Rightarrow T(n) \in \Theta(n^2)$

Observație: Ca regulă generală când stabilim clasa de complexitate, în funcția de complexitate **ignorăm termenii cu creștere mai înceată și ignorăm constanta din fața termenului dominant.**

Exerciții:

1. Demonstrați proprietățile notațiilor de complexitate:
   (a) tranzitivitatea
   (b) simetria
   (c) reflexivitatea
   (d) ...
2. Demonstrați folosind notațiile asimptotice de complexitate sau cele cu limite:
   (a) $5 * \log n + 6 \in O(n)$
   (b) $n^2 \in \Theta(n^2 + n)$
   (c) $\log_a n \in \Theta(\log_2 n)$
   (d) $\sqrt{n} \in \Omega(\log n)$
   (e) $O(n^2 + n) = O(n^2)$
3. Identificați răspunsul pentru fiecare subpunct:
   (a) $\Omega(g(n)) \cap \Theta(g(n)) = ?$

(b) $O(g(n)) \cup \Theta(g(n)) =?$

(c) $\omega(g(n)) + \Theta(g(n)) =?$

(d) $2^{2n} \in \Theta(2^n) =?$

4. Rezolvați:

   (a) Știind că f(n) $\in o(n^2)$ și h(n) $\in \Theta(n^3)$, determinați complexitatea f(n) * h(n).

   (b) Știind că f(n) $\in \Omega(n)$ și h(n) $\in O(n^2)$, determinați complexitatea h(n) / f(n).

   (c) Știind că f(n) $\in O(n^5)$ și g(n) $\in \Theta(\sqrt{n})$, este adevărată afirmația: f(g(n)) $\in \omega(n^2)$?

   (d) Știind că f(n) $\in O(g(n))$, care este limita asimptotică pentru: o(f(n)) + o(g(n))?

   (e) Știind că f(n) $\in O(\log(n))$, este adevărată afirmația: $2^{f(n)} \in$ O(n)?

5. Calculați complexitatea următoarelor bucăți de cod:

   (a)
   ```
   for (int i=1; i<c; i++) do
       //operații cu complexitate O(1)
   end for
   ```

   (b)
   ```
   for (int i=1; i<=n; i+ = c) do
       //operații cu complexitate O(1)
   end for
   ```

   (c)
   ```
   for (int i=n; i>0; i− = c) do
       for (int j=i+1; j<=n; j+ = c) do
           //operații cu complexitate O(1)
       end for
   end for
   ```

   (d)
   ```
   for (int i=n; i>0; i−−) do
       for (int j=1; j<n; j∗ =10) do
           for (int k=0; k<j; k++) do
               ... //număr constant c1 de operații
           end for
       end for
   end for
   ```

   (e)
   ```
   for (int i=1; i<n; i∗ =2) do
       for (int j=n; j>0; j/ =2) do
           for (int k=j; k<n; k++) do
               ... //număr constant c2 de operații
           end for
       end for
   end for
   ```

   (f)
   ```
   for (int i=1; i<=n; i∗ =2) do
       for (int j=0; j<i; j++) do
           for (int k=0; k<n; k+ =2) do
               ... //număr constant c3 de operații
           end for
           for (int k=1; k<n; k∗ =2) do
               ... //număr constant c4 de operații
           end for
   ```

end for
end for

Calcularea complexității algoritmilor recursivi

30 octombrie 2022

Cuprins

1 Analiza complexității algoritmilor recursivi **1**

2 Metoda iterației **4**

3 Metoda arborelui de recurență **5**

4 Exerciții **5**

1 Analiza complexității algoritmilor recursivi

Spre deosebire de un algoritm iterativ, unde putem număra operațiile efectuate în fiecare buclă, un algoritm recursiv necesită o analiză mai atentă. Un algoritm recursiv poate fi definit prin relația:

$$T(n) = cost_recursiv(n) + cost_nerecursiv(n)$$

Costul recursiv al unui apel se referă la câte noi apeluri recursive lansează un anumit apel și la dimensiunea parametrilor acestor noi apeluri, în timp ce costul nerecursiv reprezintă orice altceva mai face un apel (alte operații).

Relații de recurență uzuale:

1. Divide et impera: $T(n) = b \cdot T\left(\frac{n}{c}\right) + f(n)$
2. Chip and conquer: $T(n) = T(n-c) + f(n)$ [1]
3. Chip and be conquered: $T(n) = b \cdot T(n-c) + f(n)$ [2]

În primul rând, pentru un algoritm recursiv, trebuie definită o funcție care să descrie timpul său de rulare. Având funcția, putem aplica diverse metode pentru a determina complexitatea algoritmului.

[1] Exemplu: Algoritmul Quickselect.
[2] Exemplu: Recurența rezolvării problemei Turnurilor din Hanoi.

Vom exemplifica aceste metode prin studiul algoritmului Mergesort[1, p. 31,34].

Algoritm 1 Pseudocod Merge

```
1: funcție MERGE(A, p, q, r)
2:     n_1 = q − p + 1
3:     n_2 = r − q
4:     fie L[1..n_1 + 1] și R[1..n_2 + 1] două tablouri
5:     pentru i ← 1, n_1 execută
6:         L[i] = A[p + i − 1]
7:     final pentru
8:     pentru j ← 1, n_2 execută
9:         R[j] = A[q + j]
10:    final pentru
11:    L[n_1 + 1] = ∞
12:    R[n_2 + 1] = ∞
13:    i = j = 1
14:    pentru k ← p, r execută
15:        dacă L[i] ≤ R[j] atunci
16:            A[k] = L[i]
17:            i = i + 1
18:        altfel
19:            A[k] = R[j]
20:            j = j + 1
21:        final dacă
22:    final pentru
23: final funcție
```

Algoritm 2 Pseudocod Merge-Sort

```
1: funcție MERGE-SORT(A, p, r)
2:     dacă p ≤ r atunci
3:         q = ⌊(p + r)/2⌋
4:         MERGE-SORT(A, p, q)
5:         MERGE-SORT(A, q + 1, r)
6:         MERGE(A, p, q, r)
7:     final dacă
8: final funcție
```

Se poate observa că algoritmul *Merge* parcurge o singură dată elementele celor doi vectori, având complexitatea $\Theta(n)$. Pentru a calcula complexitatea algoritmului *MergeSort*, vom determina întâi o funcție caracteristică acestuia. Pentru a simplifica analiza complexității pseudocodul 2, vom presupune că că n, dimensiunea problemei, este o putere a lui 2. [1, cap. 2.3.2, subcap. *Analysis of merge sort*] (Algoritmul funcționează corect pentru orice n.)

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

În această formulă, n reprezintă dimensiunea vectorului primit la intrarea. Complexitatea algoritmului depinde de numărul de apeluri recursive (în cazul

nostru, 2) și de complexitatea fiecărui apel (în cazul nostru dată de apelul funcției *Merge*, având complexitatea $\Theta(n)$).

2 Metoda iterației

Recurența originală: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n;\ T(1) = c$

1. Explicităm termenul recursiv: $T\left(\frac{n}{2}\right) = 2 \cdot T\left(\frac{n}{4}\right) + c \cdot \frac{n}{2}$
2. Înmulțim cu factorul asociat: $2 \cdot T\left(\frac{n}{2}\right) = 4 \cdot T\left(\frac{n}{4}\right) + c \cdot n$
3. Repetăm până intuim forma generală pentru termenul liber.

$$
\begin{aligned}
T(n) &= 2 \cdot T\left(\frac{n}{2}\right) && +c \cdot n \\
2 \cdot T\left(\frac{n}{2}\right) &= 4 \cdot T\left(\frac{n}{4}\right) && +c \cdot n \\
4 \cdot T\left(\frac{n}{4}\right) &= 8 \cdot T\left(\frac{n}{8}\right) && +c \cdot n \\
&\dots \\
2^k \cdot T\left(\frac{n}{2^k}\right) &= 2^{k+1} \cdot T\left(\frac{n}{2^{k+1}}\right) && +c \cdot n \\
&\dots \\
2^{h-1} \cdot T\left(\frac{n}{2^{h-1}}\right) &= 2^h \cdot \underbrace{T\left(\frac{n}{2^h}\right)}_{T(1)} && +c \cdot n
\end{aligned}
$$

4. Pentru a afla adâncimea apelurilor recursive, h, trebuie egalat $T(1)$ cu forma generală găsită $T\left(\frac{n}{2^h}\right)$:

$$\frac{n}{2^h} = 1 \Rightarrow h = \log_2(n)$$

5. Explicităm pentru cazul de bază $T(1)$: $2^h \cdot T(1) = n \cdot c$
6. Adunăm toți termenii liberi:

$$T(n) = h \cdot c \cdot n + n \cdot c = (\log_2(n) + 1) \cdot c \cdot n$$

7. $T(n) \in \Theta(n \cdot \log(n))$

3 Metoda arborelui de recurență

În fig. 1 este exemplificată această metodă pentru *MergeSort*. În fiecare nod al arborelui punem complexitatea termenului liber. În partea dreaptă, sunt adunate valorile din nodurile de pe acel nivel. Suma valorilor de pe fiecare nivel va reprezenta complexitatea întregului algoritm.

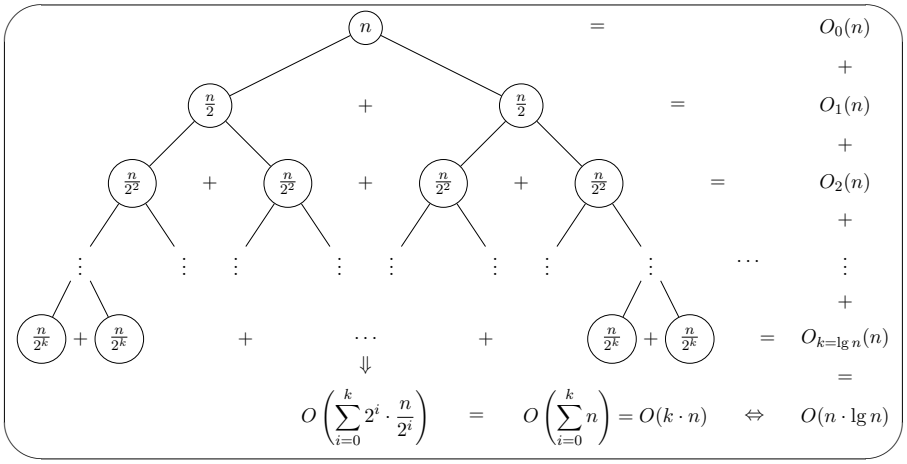


Figura 1: Arborele de recurență pentru MergeSort (aici $\lg n$ denotează $\log_2 n$)

Această metodă este foarte asemănătoare cu metoda iterației, în ambele cazuri fiind nevoiți să „ghicim" forma generală a sumei. O demonstrație formală ar necesita și o demonstrație prin inducție a aceste forme. Această reprezentare grafică poate fi mai intuitivă în unele cazuri, mai ales când metoda iterației este greu de aplicat.

4 Exerciții

1. $T(n) = 2T(n-1) + 1$
2. $T(n) = T(n-2) + \Theta(n)$
3. $T(n) = T(n-k) + \Theta(n)$, k const
4. $T(n) = T(n-a) + T(a) + \Theta(cn), a \geq 1$ și $c > 0$ const
5. $T(n) = 2T(n/3) + n \log n$
6. $T(n) = 7T(n/2) + n^2$
7. $T(n) = 2T(\sqrt{n}) + 1$
8. $T(n) = 2T(\sqrt{n}) + \log n$
9. $T(n) = 2T(\sqrt{n}) + \log(\log(n))$
10. $T(n) = T(n/5) + T(4n/5) + \Theta(n)$

11. $T(n) = T(n/5) + T(7n/10 + 6) + \Theta(n)$
12. $T(n) = T(n/2) + T(n/4) + n^2$
13. $T(n) = T(n/2 + \log(n)) + n$
14. $T(n) = 4T(n/2) + n^2/\log n$

Referințe

[1] Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C.: Introduction to Algorithms. 3rd edn. MIT Press and McGraw-Hill, USA (2009)

Metode de rezolvare a recurențelor

November 7, 2022

Contents

1 Sortarea prin interclasare (Merge-Sort) **1**

2 Metoda substituției **2**
2.1 Prezentare 2
2.2 Aplicarea metodei pentru Merge-Sort 2

3 Teorema Master **4**
3.1 Enunțul teoremei 4
3.2 Math cheatsheet 4

4 Exerciții **5**

5 Anexe **6**
5.1 Completarea demonstrației analizei complexității Merge-Sort 6

1 Sortarea prin interclasare (Merge-Sort)

Algorithm 1 Pseudocod Merge-Sort

```
1: function MERGE-SORT(A, p, r)
2:     if p ≤ r then
3:         q = ⌊(p + r)/2⌋
4:         MERGE-SORT(A, p, q)
5:         MERGE-SORT(A, q + 1, r)
6:         MERGE(A, p, q, r)
7:     end if
8: end function
```

Pentru a simplifica analiza complexității pseudocodul 1, vom presupune că n, dimensiunea problemei, este o putere a lui 2 [1, cap. 2.3.2, subcap. *Analysis of merge sort*]. (algoritmul funcționează corect pentru orice n). Recurența pentru Merge-Sort se poate rezolva ușor dacă se observă că problema inițală $T(n)$ se împarte in două subprobleme, $T(\frac{n}{2})$, la a căror complexitate se adună cea pentru `Merge()`. Funcția `Merge()` are complexitatea $\Theta(n)$ deoarece parcurge elementele celor doi vectori o singură dată. Rezultă că Merge-Sort are recurența de forma:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Folosind metoda iterației sau cea a arborilor de recurență, rezultă $T(n) \in \Theta(n \cdot log(n))$.

2 Metoda substituției

2.1 Prezentare

Metoda substituției se folosește atunci cand este posibilă intuirea complexității unei probleme care are atașată o relație de recurență. După ce se găsește soluția, aceasta trebuie verificată folosind inducția matematică. Schema pentru metoda substituției este:

- alegem soluția:
  - știm complexitatea pentru o recurență asemănătoare
  - stabilim limite inferioare și superioare din ce in ce mai apropiate de soluție
  - se folosesc celelalte metode studiate
- verificăm apartenența lui $T(n)$ la o anumită clasă de complexitate prin inducție:
  - caz de bază
  - pas de inducție de tipul $P(n) \Rightarrow P(n+1)$ sau $P(n/k) \Rightarrow P(n)$

2.2 Aplicarea metodei pentru Merge-Sort

Ne propunem să oferim o demonstrație formală a faptului că algoritmul Merge-Sort are complexitatea $\Theta(n \cdot log(n))$. Fie:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n, T(1) = 1$$

Vrem să demonstrăm că există două constante $c_1, c_2 > 0$ și un $n_0 \in \mathbb{N}_{>0}$ astfel încât:

$$P(n) : c_1 \cdot n \cdot log_2(n) \leq T(n) \leq c_2 \cdot n \cdot log_2(n), \forall n \geq n_0$$

1. **Caz de bază**

   Nu putem să alegem $n_0 = 1$ (deoarece $T(1) \neq 0$). Încercăm să demonstrăm pentru $n_0 = 2$.

   $$P(2) : c_1 \cdot 2 \cdot log_2(2) \leq T(2) \leq c_2 \cdot 2 \cdot log_2(2)$$

   Cu alte cuvinte, trebuie să arătăm că există c_1 și c_2 astfel încât: $2 \cdot c_1 \leq T(2) \leq 2 \cdot c_2$.

   Dacă înlocuim $T(2) = 2 \cdot T(1) + 2 \cdot c$, expresia devine:

   $$2 \cdot c_1 \leq 2 \cdot T(1) + 2 \cdot c \leq 2 \cdot c_2 \iff c_1 \leq T(1) + c \leq c_2$$

   $$c_1 \leq 1 + c \leq c_2 \iff \begin{cases} c_1 - 1 \leq c \\ c_2 - 1 \geq c \end{cases}$$

Știind valoarea c fixată din definiția lui T, putem opta pentru orice constantă c_1, respectiv c_2 care respectă relația anterioară.

Este important însă ca la pasul de inducție să păstrăm alegerea făcută (să folosim aceleași constante).

2. **Pasul de inducție** $P(\frac{n}{2}) \Rightarrow P(n)$

   Presupunem că $P(m)$ adevărată, pentru $\forall m \in \{2, 4, \ldots, \frac{n}{2}\}$. Pornim de la:

$$c_1 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2}) \leq T(\frac{n}{2}) \leq c_2 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2})$$

   Înmulțim inegalitatea cu 2 și adunăm c · n:

$$c_1 \cdot n \cdot \log_2(\frac{n}{2}) + c \cdot n \leq 2 \cdot T(\frac{n}{2}) + c \cdot n \leq c_2 \cdot n \cdot \log_2(\frac{n}{2}) + c \cdot n$$

$$c_1 \cdot n \cdot (\log_2(n) - 1) + c \cdot n \leq T(n) \leq c_2 \cdot n \cdot (\log_2(n) - 1) + c \cdot n$$

$$c_1 \cdot n \cdot \log_2(n) - c_1 \cdot n + c \cdot n \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) - c_2 \cdot n + c \cdot n$$

$$c_1 \cdot n \cdot \log_2(n) + n \cdot (c - c_1) \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) + n \cdot (c - c_2)$$

   Pentru a demonstra $P(n)$ trebuie să arătăm:

$$c_1 \cdot n \cdot \log_2(n) \leq c1 \cdot n \cdot \log_2(n) + n \cdot (c - c_1) \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) + n \cdot (c - c_2) \leq c_2 \cdot n \cdot \log_2(n)$$

   Cu alte cuvinte, inegalitatea este adevărată pentru orice c_1 și c_2 dacă: $c_1 \leq c \leq c_2$. În particular, putem alege $c_1 = c$ și $c_2 = c + 1$, astfel încât să respectăm toate constrângerile. Conform ipotezei de inducție, rezultă că $P(n)$ este adevărată pentru $n = 2^p, p \geq 1$.

   Rezultă că $T(n) \in \Theta(n \cdot \log(n))$. [1]

   Pentru mai multe detalii citiți 5.1

[1] Deși am analizat doar cazul când $n = 2^p, p \geq 1$, demonstrația nu își pierde din generalitate, vezi [6, p. 65], "In the case where n is not a power of c, one can usually embed a problem of size n in a problem of size n', where n' is the smallest power of c equal to or greater than n.".

3 Teorema Master

Teorema Master este o metodă generală de rezolvare a anumitor tipuri de recurențe. Putem afirma că teorema Master ne oferă o "rețetă" de rezolvare a recurențelor, însă dezavantajul constă in faptul că ea nu este aplicabilă in orice situație.

3.1 Enunțul teoremei

Fie o recurență de forma $T(n) = aT(n/b) + f(n)$, unde $a \geq 1, b \gneq 1$ constante și $f(n)$ o funcție asimptotic crescătoare pozitivă. Putem afirma că T(n) aparține unei anumite clase de complexitate, in următoarele cazuri:

1. Dacă $f(n) \in O(n^{\log_b a - \epsilon})$, pentru o constantă $\epsilon > 0$, atunci $T(n) \in \Theta(n^{\log_b a})$.

2. Dacă $f(n) \in \Theta(n^{\log_b a} \log_2^k n)$, cu $k \geq 0$, atunci $T(n) \in \Theta(n^{\log_b a} \log_2^{k+1} n)$.

3. Dacă $f(n) \in \Omega(n^{\log_b a + \epsilon})$, pentru o constantă $\epsilon > 0$ și $af(n/b) \leq cf(n)$ pentru o constantă $c < 1$ și pentru un n suficient de mare, atunci $T(n) \in \Theta(f(n))$.

Pentru a ne da seama in ce caz ne aflăm, este recomandat să comparăm $f(n)$ cu $n^{\log_b a}$:

- dacă $f(n)$ este **polinomial mai mic** decat $n^{\log_b a} \Rightarrow$ caz 1 [2]
- dacă $f(n)$ este **în aceeași clasă de complexitate cu** $n^{\log_b a}$ (eventual inmulțit cu un factor logaritmic) $\Rightarrow$ caz 2
- dacă $f(n)$ este **polinomial mai mare** decat $n^{\log_b a} \Rightarrow$ caz 3

3.2 Math cheatsheet

- $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$
- $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$
- $\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4} \approx \frac{n^4}{4}$
- $\sum_{k=1}^{n} k^\alpha \approx \frac{n^{\alpha+1}}{\alpha+1}$
- $x_{n+1} = qx_n \Rightarrow \sum_{k=1}^{n} x_k = x_0 \frac{q^{n+1}-1}{q-1}$

Pentru exemple în plus ale aplicării Teoremei Master consultați link-ul [4] din Referințe.
Pentru un cheatsheet mai elaborat consultați link-ul [5] din Referințe.

[2] Acestă afirmație necesită ca $f(n)$ să fie asimptotic mai mică decât $n^{\log_b a}$ cu un factor de n^ϵ, pentru o constantă $\epsilon \gneq 0$. [1, cap. 4.5, p. 94-95] Deci condiția, $f(n) \in o(n^{\log_b a})$, este necesară, dar nu suficientă.

4 Exerciții

1. Arătați că $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n + n\log_2 n)$.

2. Rezolvați următoarele recurențe:

- **(a)** $T(n) = 3T(2n/3) + n^3 \log_2 n$
- **(b)** $T(n) = T(n/4) + 1$
- **(c)** $T(n) = 4T(n/2) + n\sqrt{n}$
- **(d)** $T(n) = 16T(n/4) + n!$
- **(e)** $T(n) = 8T(n/3) + 2^n$
- **(f)** $T(n) = T(n/4) + lgn$
- **(g)** $T(n) = 4T(n/2) + \frac{n}{lgn}$
- **(h)** $T(n) = 3T(n/3) + nlgn$
- **(i)** $T(n) = T(n/2 - \log_2 n) + 1$

3. Demonstrați că $T(n) = 5T(n/4) + n \in \Theta(5n^{\log_4 5} - 4n)$

4. Studiați posibilitatea aplicării teoremei Master in următoarele cazuri:

- **(a)** $T(n) = 2T(n/2) + n\log_2 n$
- **(b)** $T(n) = 3T(n/3) + \frac{n}{\log_2 n}$

5. Rezolvați următoarele recurențe:

- **(a)** $T(n) = 2T(n/2) + 1$
- **(b)** $T(n) = 3T(n/3) + n/2$
- **(c)** $T(n) = 3T(n/3) + n\log_2^2 n$
- **(d)** $T(n) = 3T(n/4) + n\log_2 n$
- **(e)** $T(n) = 5T(n/4) + n$

5 Anexe

5.1 Completarea demonstrației analizei complexității Merge-Sort

Întrucât materia acestui seminar nu își propune formalizarea fiecărui pas al demonstrației, și nici vouă **NU** o să vi se ceară subtilități când veți fi evaluați, am ales să scriem aici pentru cei interesați tratarea cazului $n \neq 2^p, p \in \mathbb{N}_{>0}$

Recurența care modelează numărul de pașii efectuați de algoritmul 1, indiferent de dimensiunea problemei, este:

$S(n) = S(\lfloor n/2 \rfloor) + S(\lceil n/2 \rceil) + c \cdot n, \forall n \in \mathbb{N} \wedge n \geq 2, c > 0, S(1) = 1$

Fie $k \in \mathbf{N}$, a.î. $2^k \leq n < 2^{k+1}$. Atunci: $\frac{n}{2} < 2^k \leq n < 2^{k+1} \leq 2 \cdot n$.

Observație: $\frac{n}{2}$ poate să fie o valoare fracționară deoarece începem să lucrăm cu funcții

Pentru $\forall n = 2^p, p \in \mathbb{N}_{>0}, S(n) = T(n)$. Vom presupune S strict crescătoare, ceea ce vom și demonstra mai târziu.

$$\Rightarrow S(2^k) \leq S(n) < S(2^{k+1}) \iff T(2^k) \leq S(n) < T(2^{k+1})$$

Din cealaltă inducție:

$$c_1 \cdot 2^k \cdot \log_2(2^k) \leq S(n) < c_2 \cdot 2^{k+1} \cdot \log_2(2^{k+1})$$

Dar precum $\frac{n}{2} < 2^k, 2^{k+1} \leq 2 \cdot n$:

$$\Rightarrow c_1 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2}) \leq S(n) < c_2 \cdot 2 \cdot n \cdot \log_2(2 \cdot n)$$

$$\Rightarrow \frac{c_1}{2} \cdot n \cdot \log_2(n) - \frac{c_1}{2} \cdot n \leq S(n) < c_2 \cdot 2 \cdot n + c_2 \cdot 2 \cdot n \cdot \log_2(n)$$

Vom folosi inegalitatea: $n \leq \frac{n}{2} \cdot \log_2(n), \forall n \geq 4$.

Putem găsi o expresie mai mică decât termenul din stânga înlocuindu-l pe $\frac{c_1}{2} \cdot n$ cu $\frac{c_1}{4} \cdot n \cdot \log_2(n)$, inegalitate adevărată pentru un n suficient de mare. Analog, pe $c_2 \cdot 2 \cdot n$ îl putem înlocui cu $c_2 \cdot 2 \cdot n \cdot log(n)$.

$$\Rightarrow \frac{c_1}{4} \cdot n \cdot \log_2(n) \leq S(n) < 4 \cdot c_2 \cdot n \cdot \log_2(n), \forall n \geq n_0, n_0 \text{ fixat natural}$$

Reamintindu-ne definiția clasei Θ, am găsit două constante, $c' = \frac{c_1}{4}$ și $c'' = 4c_2$, astfel încât $n \cdot \log_2(n)$ este o margine asimptotic tare a lui $S(n)$.

$$\Rightarrow S(n) = \Theta(n \cdot \log(n))$$

A mai rămas să demonstrăm că S e o funcție strict crescătoare. Vom face asta prin inducție, demonstrând proprietatea:

$$Q(n) : T(n-1) \leq T(n), \forall n \in \mathbb{N}$$

Baza inducției: $S(1)$ o constantă. $S(2) = 2S(1) + 2c$. Avem $T(1) < T(2)$, deci $Q(2)$ adevărată.
Pasul inducției: Presupunem $Q(h), \forall h < n$ adevarată. Arătăm că: $Q(n-1) \rightarrow Q(n)$.

Caz 1 Dacă n este par, $n = 2k, k \in \mathbb{N}$.

$$Q(n) \Leftrightarrow S(n-1) \leq S(n)$$

$$Q(n) \Leftrightarrow S(k-1) + S(k) + c(2k-1) \leq 2S(k) + c(2k)$$

Ceea ce este adevarat din ipoteza inductivă și pentru că $c > 0$.

Caz 2 Daca n este impar, avem $n = 2k+1, k \in \mathbb{N}$.

$$Q(n) \Leftrightarrow S(n-1) \leq S(n)$$

$$Q(n) \Leftrightarrow 2S(k) + c(2k) \leq S(k) + S(k+1) + c(2k+1)$$

$$Q(n) \Leftrightarrow S(k) \leq S(k+1) + c$$

Ceea ce este adevarat din ipoteza inductivă și pentru că $c > 0$.

Astfel, am demonstrat că $Q(n)$ este adevărată, $\forall n \in \mathbb{N}$.

Vă recomandăm să citiți și alte demonstrații, precum [2] și [3].

Referințe

[1] Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C.: Introduction to Algorithms. 3rd edn. MIT Press and McGraw-Hill, USA (2009)

[2] CS 3110 Lecture 19, Substitution method for recurrence relations, https://www.cs.cornell.edu/courses/cs3110/2008fa/lectures/lec19.html. Ultima accesare la 6 Noi 2021

[3] Open Data Structures, Chap. 11.1, Comparison-Based Sorting, http://opendatastructures.org/versions/edition-0.1e/ods-java/11_1_Comparison_Based_Sorti.html. Ultima accesare la 6 Noi 2021

[4] Master Theorem: Practice Problems and Solutions, https://www.csd.uwo.ca/~mmorenom/CS433-CS9624/Resources/master.pdf. Ultima accesare la 6 Noi 2021

[5] Algorithm Analysis – Math CheatSheet, https://raw.githubusercontent.com/AAResources/resources/main/Math_Cheatsheet___Algorithm_Analysis.pdf. Ultima accesare la 6 Noi 2021

[6] Aho, Alfred V and Hopcroft, John E: The design and analysis of computer algorithms, Pearson Education India, 1974.

ANALIZĂ AMORTIZATĂ

Autor original: Darius-Florentin Neațu
Editare: Liliana Grigoriu (2021)

Noiembrie 2022

CUPRINS

1 Analiza amortizată 2
1.1 De ce o studiem? . . . 2
1.2 Ce este analiza amortizată? . . . 2
1.3 Ce NU este analiza amortizată? . . . 2
2 Contor binar (Binary counter) 4
3 Tehnici de calcul 4
3.1 Analiza agregată (aggregate analysis) . . . 5
3.1.1 Contor binar . . . 5
3.2 Metoda creditelor (accounting method) . . . 6
3.2.1 Contor binar . . . 6
3.3 Metoda potențialului (potential method) . . . 7
3.3.1 Contor binar . . . 8
4 Exerciții propuse 10
4.1 O problemă oarecare . . . 10
4.2 Stivă cu backup (backup stack) . . . 10
4.3 Vector redimensionabil (resizable array) . . . 10
4.4 Vector extensibil (extendable array) . . . 11
4.5 O structură de date nouă . . . 11

1 ANALIZA AMORTIZATĂ

[1]

1.1 De ce o studiem?

Există operații, aplicate asupra unor structuri de date, a căror timp de execuție variază foarte mult pe parcursul unui program. De exemplu, este posibil ca operația aplicată să fie ineficientă într-o anumită situație, care să apară însă relativ rar pe parcursul execuției programului. In acest context este interesant să investigăm timpul de execuție pentru întreaga secventă de operații de același tip din algoritm.

La structuri de date ați implementat **hash tables** care permiteau permiteau efectuarea eficientă, în $O(1)$, a operațiilor de căutare si inserare de elemente. Această afirmație se bazează pe presupunerea că numărul de elemente stocate este comparabil cu numărul de buckets (liste/vectori), altfel complexitatea pentru căutare este $O(n)$. În practică, dacă condițiile de mai sus sunt îndeplinite, vom spune ca implementarea de hash table are $O(1)$ **costul amortizat** pentru operațiile de cautare, respectiv inserare.

1.2 Ce este analiza amortizată?

Definiție 1. Analiza amortizată este o metoda de a analiza o secvență de n operatii, din perspectiva resurselor consumate (timp/memorie), pentru cazul cel mai defavorabil.

Folosind analiza amortizată, putem demonstra că pentru o operație individuală din secvență costul mediu este mic, chiar dacă unele operații din secvență pot fi costisitoare.

Analiza amortizată diferă de analiza medie probabilistică (average-case analysis) prin faptul că rezultatele sunt **exacte** (noțiunea de probabilitate nu este inclusă). Analiza amortizată **garantează** că performanța medie pe operație în cazul cel mai defavorabil este cea calculată.

1.3 Ce NU este analiza amortizată?

NU este o tehnică de design pentru algoritmi. Dacă vi se dă o problemă, nu începeți prin a spune: "Pot să aplic tiparul general al unui algoritm de analiză amortizată și să îl modific pentru a obține soluție la problema mea." **NU** putem folosi analiza amortizată pentru a proiecta un algoritm.

NU este o modalitate de a reface un algoritm existent. Analiza amortizată nu schimbă în nici un fel algoritmul, ci este doar un mod mai precis de evaluare a

[1] Un studiu al acestei tehnici poate fi urmărit în articolul lui Tarjan [1].

timpului cel mai defavorabil de execuție a unei secvențe de operații (mult mai precis decât dacă am însuma toți timpii worst-case pentru toate operațiile din secvență).

2 CONTOR BINAR (BINARY COUNTER)

Fie un contor binar pe k biți, care numără crescător începând de la 0. Vom folosi notația $A[0..k-1]$ pentru a descrie un șir de k biți, cu precizarea că bitul cel mai puțin semnificativ (least significant bit - LSB) este $A[0]$, iar bitul cel mai semnificativ (most significant bit - MSB) este $A[k-1]$.

Dacă x este numărul curent arătat de contor, atunci $x = \sum_{i=0}^{k-1} A[i] * 2^i$. Inițial avem $x = 0$, deci $A[i] = 0$.

Pentru a adăuga 1 (modulo 2^k) la valoarea contorului, vom folosi următorul pseudocod.

```
function Inc(A) {
        // i = rangul binar curent
        i = 0;

        // daca adun 1, trebuie sa propag transportul
        // pana cand gasesc un zero
        // 0 + 1 = 1 si nu mai am transport
        while (i < A.length && A[i] == 1) {
                A[i] = 0;
                i++;
        }

        // am gasit un zero daca nu am facut overflow
        if (i < A.length) {
                A[i] = 1;
        }
}
```

Care este costul amortizat pentru o operație de incrementare?

3 TEHNICI DE CALCUL

Analiza amortizată este un subiect acoperit în multe publicații. Recomandăm să consultați "Introduction to algorithms" [2], respectiv "Introducere în Analiza Algoritmilor" [3]. Acest concept este explicat pe scurt și în "Algorithm Design: Foundations, Analysis and Internet Examples" [4].

Cele mai comune 3 tehnici folosite în analiza amortizată sunt: analiza agregată, metoda creditelor și metoda potențialului.

3.1 Analiza agregată (aggregate analysis)

În **analiza agregată**, vom arăta că pentru oricare secvență de **n** operații, timpul necesar (numărul de pași) în cel mai rău caz (**worst-case time**) este în total $T(n)$. Această valoare este determinată în funcție de specificul algoritmului studiat.

Definiție 2 (Cost amortizat - metoda agregată). În cel mai rău caz, costul mediu sau **costul amortizat** pentru fiecare operație este $\frac{T(n)}{n}$.

ATENȚIE: Acest cost amortizat se aplică fiecărei operații din secvență, chiar dacă sunt mai multe tipuri de operații, iar unele au cost real mai mare. Celelate două metode studiate pot asocia costuri diferite pentru fiecare operație.

3.1.1 *Contor binar*

Observăm că atunci când facem o operație de **Inc** nu toți biții se modifică (se ia o decizie și se stabilește care biți se modfică la fiecare pas).

- $A[0]$ se modifică după fiecare iterație $\rightarrow$ din 2^0 în 2^0 pași
- $A[1]$ se modifică după fiecare două iterații $\rightarrow$ din 2^1 în 2^1 pași
- $A[2]$ se modifică după fiecare patru iterații $\rightarrow$ din 2^2 în 2^2 pași
- ....
- $A[j]$ se modifică după 2^j incrementări

Pentru a calcula numărul total de operații rescriem astfel:

- $A[0]$ se modifică de $\frac{n}{2^0}$ ori
- $A[1]$ se modifică de $\frac{n}{2^1}$ ori
- $A[2]$ se modifică de $\frac{n}{2^2}$ ori
- ....
- $A[j]$ se modifică de $\frac{n}{2^j}$ ori

Adunând precedentele k relații ($0 \leqslant j \leqslant k-1$), obținem:

$$T(n) = \sum_{j=0}^{k-1} \frac{n}{2^j} = n \sum_{j=0}^{k-1} \frac{1}{2^j} < n \sum_{j=0}^{\infty} \frac{1}{2^j} = 2n$$

Prin urmare, **costul total amortizat** este $T(n) \leqslant 2n$, adică $T(n) = O(n)$.
Pentru o operație **costul amortizat** este $\frac{T(n)}{n} \leqslant 2$, adică $O(1)$.

3.2 Metoda creditelor (accounting method)

În tehnica de analiză amortizată numită **metoda creditelor**, asignăm diferite costuri fiecărei operații, unele având un cost mai mic sau mai mare față de costul real.

Definiție 3 (Cost amortizat - metoda creditelor). Costul amortizat al unei operații este suma (cantiatea) cu care a fost taxată o operație.

Dacă costul amortizat al unei operații depășește costul real, diferența o retinem într-o structură specială numită **credit**. Creditele ne ajută să compensăm operațiile ulterioare al căror cost amortizat este mai mic decât costul real.

Putem vedea costul amortizat al unei operații ca fiind împărțit între costul actual și credit (care poate fi depozitat sau folosit).

Operații diferite pot avea cost amortizat diferit. Trebuie **să alegem** costul amortizat al unei operații cu foarte mare atenție. Dacă vrem să arătăm, folosind această tehnică, că în cel mai rău caz costul mediu pe operație este mic, trebuie să ne asigurăm că întotdeauna costul total amortizat al secvenței de operații este o limită superioară a costului total real.

Pentru orice secvență de n operații obținem:

$$\sum_{i=1}^{n} \hat{c}_i \geqslant \sum_{i=1}^{n} c_i$$

- n = numărul de operații din secvența dată
- c_i = **costul real** al operației cu numărul i din secvență
- $\hat{c}_i$ = **costul amortizat** al operației cu numărul i din secvență

Cu aceste notații putem defini și creditul pentru o operație:

- $credit_i$ = creditul asociat operației i
- $credit_i = \hat{c}_i - c_i$
- $credit_i \geqslant 0 \iff$ "depunem" bani în cont
- $credit_i < 0 \iff$ "retragem" bani din cont

Creditul total acumulat în structura de date este diferența între costul total amortizat și costul total real. Acesta trebuie să fie mereu (pentru orice n) pozitiv, pentru că nu putem cheltui mai mult decât avem.

$$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i \geqslant 0$$

3.2.1 *Contor binar*

Considerăm același contor binar pe k biți despre care s-a vorbit anterior.

O abordare diferită față de metoda anterioară, este să atribuim costuri fiecărei schimbări de bit (sarcini) din cadrul unei operații. [5, vezi "Taxation Method 1"]

Facem observația că operația **Inc** schimbă doar starea unor biți (tranziții 0 în 1 și 1 în 0). Astfel definim următoarele sarcini:

- **set** = reprezintă o tranziție din 0 **în** 1 a unui bit (oarecare)
- **reset** = reprezintă o tranziție din 1 **în** 0 a unui bit (oarecare)

Pentru fiecare sarcină, set sau reset, costul real este 1.
În ideea că plătim în avans pentru sarcinile de reset când au loc sarcinile de set, alegem costuri amortizate de 2 pentru set și de 0 pentru reset.

Sarcină	Costul real	Costul amortizat
set	1	2
reset	1	0

Operația **Inc** schimbă un singur bit de 0 în 1 (face un singur set), iar toți biții mai puțin semnificativi decât acesta, sunt făcuți 0 (se aplică pe toți reset).

De fiecare dată când fac un **set**, ar trebui să platesc 2 credite (costul amortizat ales de mine), dar costul real este 1, astfel încât consum un credit pentru sarcina efectivă de set, iar un credit îl pun în cont; acest credit va fi folosit atunci când același bit va trece din 1 în 0 (reset).

Se observă că, în fiecare moment, în cont există un credit pentru fiecare bit de 1 din număr. Putem afirma că nu plătim nimic în plus pentru un reset (fiecare bit de 1 are creditul lui deja - "își plătește singur"). Prin urmare, costul de resetare, care este plătit în while, este acoperit de creditele din cont.

Deoarece numărul de credite din cont este egal cu numărul de biți de 1 din număr, iar numărul nu poate avea "un număr negativ de biți de 1" , rezultă mereu că în cont avem o sumă nenegativă. Deci **am ales corect** costurile amortizate pentru cele două sarcini.

O operație **Inc** este compusă din maximum o sarcină de set (la overflow nu face set - putem spune că vom reseta contorul) și câteva sarcini de reset. Cele din urmă au cost amortizat 0, în consecință o operație de **Inc** are costul amortizat cel mult 2.

Evident, obținem aceeași concluzie: pentru o operație **costul amortizat** este $O(1)$.

3.3 Metoda potențialului (potential method)

În loc să reprezentăm munca prevăzută ca și credite acumulate, **metoda potențialului** de analiză amortizată reprezintă munca ca "energie potențială" sau "potențial", care poate fi eliberată pentru a plăti operații. Asociem potențialul mai degrabă cu structura de date ca și un întreg (nu mai considerăm obiectele ca fiind individuale).

Vom face următoarele notații:

- D_0 = structura de date inițială, pe care vom executa n operații

- c_i = **costul real** al operației i
- D_i = **structura de date** care rezultă după aplicarea operației i asupra structurii de date D_{i-1}
- $\Phi(D_i)$ = **funcție de potențial** care asociază fiecărei structuri D_i un număr real, numit potențialul ("energia") structurii de date
- $\hat{c}_i$ = **costul amortizat** al operației i în raport cu funcția de potențial dată

Asemănator legilor fizicii, variația de potențial este cea care modifică costul ("starea") curent: costul amortizat al unei operații este costul real la care se adaugă variația de potențial.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Însumând cele n egalități obținem relația care ne arată dependența între costul total amortizat și costul total real.

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$$

Precedenta relație ne arată că dacă vrem să majorăm costul real folosind costul amortizat atunci $\Phi(D_n) \geqslant \Phi(D_0)$.

În practică nu știm întotdeauna de la început câte operații se vor executa. Dacă impunem ca $\Phi(D_i) \geqslant \Phi(D_0), \forall i \leqslant n$, atunci garantăm că, la fel ca la metoda creditelor, plătim în avans.

De obicei, alegem $\Phi(D_0) = 0$ și doar demonstrăm că $\Phi(D_i) \geqslant 0$

Definiție 4 (Cost amortizat - metoda potențialului). Dacă s-a definit o funcție de potențial, pentru structura de date, care este inițial zero și întotdeauna nenegativ, costul amortizat al unei operații este costul real la care se adaugă diferența de potențial.

3.3.1 *Contor binar*

Pentru a alege o funcție de potențial pentru problema contorului, ne gândim cum "curge energia". Se pornește de la 0, se tot adaugă 1 și numărul de biți de 1 crește ("energia crește"), iar din când în când (mai exact cand numărul curent este de forma $2^{length} - 1$), toți biții se resetează, deci devin 0, ("energia este consumată") și se rămâne cu un singur bit de 1 (cel setat in aceasta operație).

Observând această analogie, alegem funcția de potențial

$$\Phi(D_i) = b_i$$

În continuare voi descrie notațiile folosite.

- $\Phi(D_i)$ = potențialul contorului după aplicarea operației cu numărul i
- $\Phi(D_0)$ = potențialul inițial al contorului (înainte de prima operație)

- b_i = numărul de biți de 1 după aplicarea operației cu numărul i
- t_i = numărul de biți de 1 care sunt facuți 0 de către operația i
- c_i = costul real al operației i
- $\hat{c}_i$ = costul amortizat al operației i

Ca să putem aplica metoda, funcția aleasă de noi trebuie se respecte condiția

$$\Phi(D_i) \geqslant \Phi(D_0)$$

.

Presupunem că numărătoarea începe de la 0, deci $b_0 = 0$, deci și $\Phi(D_0) = 0$. Atunci este suficient să arătăm că $\Phi(D_i) \geqslant 0$. Condiția este mereu verificată întrucât mereu avem un număr nenegativ de biți de 1.

Pentru a calcula costul amortizat aplicăm formula menționată în teorie.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Deoarece operația i resetează t_i biți, apoi face un set, costul real al operației i este

$$c_i = 1 + t_i$$

(durează t_i unități să reseteze t_i biți; durează 1 unitate să seteze 1 bit).

În continuare calculăm variația de potențial.

Numărul de biți de 1 de la pasul curent, este numărul de biți de la pasul anterior, din care scădem numărul de biți resetați la pasul curent, apoi adăugăm și bitul setat la acest pas.

$$b_i = b_{i-1} - t_i + 1$$

Relația anterioară este valabilă doar când $b_i > 0$. Dacă $b_i = 0$ ($i > 0$), înseamnă că la pasul anterior toți cei k biți erau 1 ($b_{i-1} = k$) și la pasul curent toți au fost făcuți 0 ($t_i = k$). Nu se mai setează bit la acest pas (vezi condiția de la finalul ciclului, contorul devine 0 în caz de overflow). În acest caz particular avem

$$b_i = b_{i-1} - t_i < b_{i-1} - t_i + 1$$

Pentru o operație oarecare i, avem următoarea relație

$$b_i \leqslant b_{i-1} - t_i + 1 \iff \Phi(D_i) \leqslant \Phi(D_{i-1}) - t_i + 1$$

Din precedenta relație putem scoate variația de potențial (necesară în formula de cost amortizat).

$$\Phi(D_i) - \Phi(D_{i-1}) \leqslant 1 - t_i$$

Înlocuim rezultatele obținute pentru a afla costul amortizat.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = (1 + t_i) + \Phi(D_i) - \Phi(D_{i-1}) \leqslant (1 + t_i) + (1 - t_i) = 2$$

Costul amortizat pentru operația **Inc** este 2. Prin urmare aceasta are cost O(1) amortizat.

4 EXERCIȚII PROPUSE

4.1 O problemă oarecare

Să presupunem că avem de efectuat o succesiune de n operații pe o structură de date în care operația cu numărul i are asociat costul c_i de mai jos.

$$c_i = \begin{cases} i, & \text{dacă i este putere a lui 2} \\ 1, & \text{altfel} \end{cases}$$

Folosiți metoda agregării și/sau metoda creditelor pentru a determina costul amortizat per operație.

4.2 Stivă cu backup (backup stack)

Să presupunem că avem de efectuat o succesiune de n operații pe o stivă a cărei dimensiune niciodată nu depașeste o valoare k. Costul real al acestor operații (push, pop) este constant, si il consideram ca fiind 1. După fiecare k operații, facem o copie (backup) întregii stive (copierea unui element are cost 1 in cadrul acestei operații).

Calculați, folosind metoda creditelor, costul a n operații pe stivă, incluzând operațiile de backup.

Hint: Se poate obține costul total $O(n)$ dacă se aleg costurile amortizate corespunzătoare.

4.3 Vector redimensionabil (resizable array)

Un vector redimensionabil este o structură de date care stochează o secvență de elemente și suportă operațiile de mai jos.

- **AddToEnd(D, x)**: adaugă elementul x la sfârșitul secvenței
- **LookUp(D, k)**: returnează al k-lea element din secvență sau NULL dacă lungimea curentă a secvenței este mai mică decât k

La adaugarea unui element când capacitatea actuală n a vectorului nu are loc pentru elementul respectiv, se rezerva un spatiu nou pentru vector cu capacitate dubla $2n$, și se muta elementele existente în primele n locuri ale noului spatiu, cauzând cost n. Apoi se eliberează spațiul folosit anterior de vector. La început presupunem că vectorul are spațiu pentru un element. Dacă există loc în vector presupunem că accesarea locului următor ultimului element și așezarea elementului nou pe acest loc ia timp 1 sau $O(1)$. Care este costul amortizat pentru operația **AddToEnd**?

4.4 Vector extensibil (extendable array)

Un vector extensibil este o structură de date care stochează o secvență de elemente și suportă operațiile de mai jos.

- **AddToFront(D, x)**: adaugă elementul x la începutul secvenței
- **AddToEnd(D, x)**: adaugă elementul x la sfârșitul secvenței
- **LookUp(D, k)**: returnează al k-lea element din secvență sau NULL dacă lungimea curentă a secvenței este mai mică decât k

Descrieți o simplă structură de date care implementează algoritmul cerut. Primele două operații trebuie să aibă un cost amortizat $O(1)$, operația **LookUp** are worst-case O(1). De asemenea, structura voastră ar trebui să ocupe un spațiu $\theta(n)$, unde n este lungimea curentă a numărului de element din subsecvență.

4.5 O structură de date nouă

Folosind ideea de la Insertion Sort vrem să construim o structură de date care să susțină executarea în mod eficient a operațiilor SEARCH si INSERT.

Operația de SEARCH se execută în $O(\log(n))$ pe un vector sortat (folosing căutare binară). Din păcate, operația INSERT rulează în $O(n)$, întrucât în cel mai defavorabil caz suntem nevoiți să shiftăm toate elementele pentru a face loc pentru cel nou.

Presupunem că dorim să suportăm o secvență de n operații SEARCH și INSERT. Fie $< n_{k-1}n_{k-2}...n_0 >$ reprezentarea binară a lui n (unde $k = [\log(n+1)]$).

În loc să folosim un singur vector sortat de dimensiune n (numit **v**), vom folosi k vectori sortați, numiti $v_0, v_1, ..., v_{k-1}$, care respectă relația următoare.

$$|v_i| = \begin{cases} 2^i, & \text{dacă } n_i = 1 \\ 0, & \text{dacă } n_i = 0 \end{cases}$$

Se observă că, deși fiecare vector este sortat, **nu** există o relație particulară între elementele din vectori diferiți.

Operația de SEARCH se realizează făcând căutare binară.

Operația de INSERT se realizează ca în următorul pseudocod, complexitatea fiind $O(n)$.

```
function Insert(A[1..k], x) {
        // vector care contine doar elementul
        newArray = {x};

        i = 0;
        while (A[i] is NOT empty) {
                // apeleaza subrutina de la Merge Sort
                newArray = merge(A[i], newArray);
```

```
                A[i] = empty;
                i++;
        }

        A[i] = newArray;
}
```

Se cere să se determine raspunsul pentru următoarele cerințe:

- Complexitatea operației SEARCH (worst-case).
- Costul amortizat pentru o secvență de n operații de tip INSERT, utilizând metoda potențialului.
- Dați exemple de structuri de date învățate care oferă un timp de rulare mai bun pentru ambele operații.

Hint: $\Phi(D_i) = \sum_{j=0}^{k-1} (k-j) n_j 2^j$

BIBLIOGRAFIE

[1] Robert Endre TARJAN. Amortized computational complexity. 1985.

[2] Charles E Leiserson, Ronald L Rivest, Clifford Stein, and Thomas H Cormen. *Introduction to algorithms*. MIT press, 2009.

[3] Cristian A Giumale. *Introducere în analiza algoritmilor: teorie şi aplicaţie*. Polirom, 2004.

[4] Michael T Goodrich and Roberto Tamassia. *Algorithm design: foundation, analysis and internet examples*. John Wiley & Sons, 2006.

[5] Jeff Erickson. http://jeffe.cs.illinois.edu/teaching/algorithms/notes/09-amortize.pdf, 2018.

Algoritmi nedeterminiști

November 24, 2022

1 Clasificare

Algoritmii se pot clasifica în:

- **Determiniști**:
  - fiecare acțiune/operație are un rezultat unic determinat
  - **serialitate**: pentru orice moment de timp t din cursul execuției există o singură acțiune efectuată la momentul t
- **Nedeterminiști**:
  - există acțiuni/operații al căror rezultat nu este unic definit ci are valori într-o mulțime finită de posibilități
  - **paralelism**: structură arborescentă de operații, unde operațiile de pe o cale din arbore sunt efectuate serial, în timp ce operațiile de pe căi diferite sunt efectuate în paralel. La un moment de timp t se execută mai multe acțiuni pe diverse căi.
  - nu are implementare practică

2 Operații specifice

- choice(A)
  - ramifică copia curentă a algoritmului în n copii, unde n = card(A)
  - A = mulțime **finită**!
  - pentru fiecare valoare din A se creează o copie a algoritmului care continuă cu acea valoare
  - variabilele locale ale algoritmului sunt clonate pentru fiecare copie
  - copiile continuă în paralel și independent una de alta
- fail - dacă o copie se termină cu fail (insucces), atunci se termina execuția acelei copii; restul copiilor continuă execuția

- success - dacă o copie se termină cu success, atunci se termină execuția întregului program (practic execuția întregului algoritm se termină cu succes).

Observații

- **Valorea de return a algoritmului**:
  - success (când una din căi se termină cu success, nu contează restul)
  - fail (când toate căile se termină cu fail)
- **Algoritmii de optim** pot fi modelați ca apeluri succesive ale unor **algoritmi de decizie** (de exemplu determinarea arborelui de acoperire minim pe un arbore fără costuri pe muchii: există arbore de acoperire din 1 muchie? dacă nu, există din 2 muchii? etc)

3 Complexitate

Complexitatea temporală a unui algoritm nedeterminist (se mai numește și **complexitate angelică**):

- suma complexităților operațiilor din secvența/calea cea mai scurtă care termină algoritmul cu success
- dacă toate căile întorc fail, complexitatea este suma complexităților de pe calea cea mai lungă încheiată cu fail
- **Obs**: choice(A) are complexitate O(1).

4 Etape

Etape în execuția unui algoritm nedeterminist:

- **generare** (choice – generează câte o copie pentru fiecare candidat la a fi soluție) – aceasta este partea nedeterministă a algoritmului
- **testare** (fiecare candidat generat este testat dacă e o soluție corectă)

5 Exemple de algoritmi nedeterminiști

5.1 Căutarea unui element într-un vector

```
// V = vectorul, n = nr de elemente din vector, e = elementul cautat
caut(V, n, e) {
        i = choice(1 .. n)  //generare - pozitia elementului cautat
        if (V[i] = e)
                success  //testare - este elementul cautat?
        fail
}
```

5.2 Test dacă un număr natural este neprim

```
// n = numarul testat
neprim(n) {
        i = choice(2 .. floor(sqrt(n))) // generare - posibilii divizori
        if (n mod i == 0) // testare - este i divizor pentru n?
                success
        fail
}
```

5.3 Sortarea unui vector de elemente strict pozitive

```
// V = vectorul, n = nr de elemente din vector
sort(V, n) {
        for i = 1 .. n
                Aux[i] = 0 // in Aux vom crea vectorul sortat
        for i = 1 .. n {
                j = choice(1 .. n) // generarea pozitiei ocupata de v[i]
                if (Aux[j] > 0) // testare - nu e deja ocupata?
                        fail
                Aux[j] = V[i]
        }
        for i = 1 .. (n - 1)
                if (Aux[i] > Aux[i + 1]) // testare - e sortat?
                        fail
        success // daca niciun test nu a dat fail, success
```

Obs:
P = PTIME = clasa problemelor rezolvabile prin algoritmi determiniști polinomiali
NP = NPTIME = clasa problemelor rezolvabile prin algoritmi nedeterminiști polinomiali

Obs:
NP nu înseamnă ca nu este P! NP vine de la "non deterministic polynomial time", nu de la "not P". In fapt, P $\subseteq$ NP (orice algoritm determinist polinomial poate fi ușor transformat într-un algoritm nedeterminist polinomial), iar dacă P = NP rămâne în continuare o problemă deschisă (cel mai plauzibil este că nu sunt egale, însă nu s-a putut demonstra încă).

Problemă:

- **tractabilă:** rezolvabilă printr-un algoritm determinist polinomial
- **intractabilă:** toți algoritmii determiniști care o rezolvă sunt supra-polinomiali

6 Exerciții

Găsiți un algoritm nedeterminist pentru următoarele probleme și calculați complexitatea în fiecare caz:

1. Fiind dat un vector de numere, există o subsecvență de elemente egale consecutive de lungime mai mare decât **k**?
2. Având un graf, să se determine dacă există un drum de la nodul **u** la nodul **v** care are lungimea mai mică decât o valoare dată **dim**.
3. Colorarea unui graf:
   Dându-se un graf G(V, E) și **k** culori, se pot colora nodurile grafului doar cu cele **k** culori astfel încât niciun nod să nu aibă un vecin de aceeași culoare?
4. k-clică:
   Dându-se un graf G(V, E) și un număr **k**, există un subgraf complet (o clică) de dimensiune **k**?
5. k-acoperire (vertex cover): Dându-se un graf G(V, E) și un număr **k**, există o submulțime de **k** noduri astfel încât fiecare muchie (v1,v2) să aibă cel puțin unul dintre nodurile care o compun (v1 sau v2) în submulțimea aleasă?

6. Submulțime de sumă dată (Q-sume):
   Se dă o mulțime de N numere și un număr Q. Există o submulțime de numere a căror sumă să fie fix Q?

7. Problema comis-voiajorului (TSP):
   Se dă o mulțime de orașe conectate între ele prin drumuri. Există vreo modalitate ca un comis voiajor să viziteze toate orașele o singură dată și să se întoarcă de unde a plecat?

8. Plasați 8 regine pe o tablă de șah fără ca acestea să se atace.

9. Problema subgrafurilor izomorfe:
   Două grafuri G1(V1, E1) și G2(V2, E2) sunt izomorfe dacă există o funcție bijectivă $f : V1 -> V2$ astfel încât: muchia (u, v) este în E1 <=> muchia (f(u), f(v)) este în E2 (obs: două grafuri pot fi izomorfe dacă au același număr de noduri). Dându-se două grafuri, G1 și G2, există un subgraf în G1 care să fie izomorf cu G2?

10. Independent set:
    Dându-se un graf G(V, E) și un număr k din Z, există o mulțime S de k noduri astfel încât orice muchie are cel mult un capăt în S?

11. Problema partiționării:
    Dându-se o mulțime de t numere întregi, există o împarțire a elementelor sale în două submulțimi S1 și S2 care să aibă sume egale?

12. SAT(Boolean Satisfiability Problem):
    Se dă o expresie booleană în forma normală conjunctivă - o conjuncție de clauze, unde clauzele sunt disjuncții. Exemplu $(x1 \vee \neg x2) \wedge (\neg x1 \vee x2 \vee x3) \wedge \neg x1$. Să se determine dacă există o posibilitate de atribuire a variabilelor astfel încât expresia să fie adevărată.

Reduceri polinomiale

11 decembrie 2022

Numim **instanțe-da (engl. yes-instance)** ale unei probleme de decizie P instanțele (datele de intrare) pentru P care apartin multimii definite de P. De exemplu un triunghi este o instanța-da pentru problema 3-CLIQUE (care primeste ca input un graf si accepta in cazul in care graful respectiv contine 3 noduri cu muchii intre fiecare 2 dintre ele). Un graf care nu contine nici un triunghi este o **instanța-nu** pentru problema 3-CLIQUE.

Reducere polinomială

PA: o problema pe care o reduc

PB: o problema la care reduc problema PA

F = algoritm **determinist polinomial** de transformare a instantelor problemei PA in instanța ale problemei PB Dacă există F algoritm determinist cu complexitate polinomială care transformă **orice** instantă I a problemei PA într-**o** instanță F(i) a problemei PB a.î

I este o instanța-da pentru PA <=> F(I) este o instanța-da pentru PB,

atunci spunem că PA se reduce polinomial la PB: **PA** $\leq_p$ **PB**.

Obs0: diferența dintre reducerea polinomială și reducerea Turing este că aici F trebuie să fie determinist polinomial.

Obs1: faptul ca reducerea nu merge (neapărat) și de la PB la PA se vede în sublinierea cuvintelor "orice" și "o". Când reducem PA la PB luăm o intrare oarecare pentru PA și construim în mod convenabil o intrare pentru PB.

Obs2: demonstrația faptului ca o problemă se reduce polinomial la alta nu e completă dacă nu demonstrați **implicația în ambele sensuri**. Întâi trebuie demonstrat că daca I este o instanța-da pentru PA atunci F(I) este o instanța-da pentru PB, apoi ca daca F(I))este o instanța-da pentru PB atunci I este o instanța da pentru PA.

Consecințe ale stabilirii unei relații de reducere polinomială:

$A \leq_p B \wedge B \in P \Rightarrow A \in P$

$A \leq_p B \wedge A \notin P \Rightarrow B \notin P$

Aceste consecințe se folosesc pentru a demonstra apartenența unor probleme la o anumită clasă de probleme, folosind reduceri (de) la probleme a căror (in)tractabilitate este cunoscută.

Clase de probleme

P = { A | $\exists$ B un algoritm determinist polinomial care rezolvă A }

NP = { A | $\exists$ B un algoritm nedeterminist polinomial care rezolvă A }

NP-hard = { A | $\forall$ Q $\in$ NP $\bullet$ Q $\leq_p$ A }

NP-complete = { A | A $\in$ NP $\wedge$ A$\in$ NP-hard }

Strategii de a demonstra apartenența la o clasă de probleme

P: se construiește un algoritm determinist polinomial

NP: se construiește un algoritm nedeterminist polinomial SAU se arată ca soluția poate fi verificată în timp polinomial

NP-hard: se găsește o problema cunoscută ca NP-hard și se reduce această problemă la problema curentă (nu invers!)

NP-complete: se arată că e NP si NP-hard

Alte clase de probleme

PSPACE = clasa problemelor care sunt rezolvate de algoritmi deterministi in timp nelimitat si utilizând spațiu polinomial

NPSPACE = clasa problemelor care sunt rezolvate de algoritmi NEdeterministi în timp nelimitat și utilizând spațiu polinomial

PSPACE = NPSPACE (Savitch, 1970)

Teorema 1. Relația de reducere polinomială $\leq_p$ este tranzitivă.

Teorema 2. O problemă Q2 este NP-hard dacă și numai dacă există o problema NP-completă Q1 astfel încat Q1 $\leq_p$ Q2.

Teorema 3. Problemele NP-complete formează o clasă de echivalență în raport cu relația $\leq_p$ (altfel spus, dacă Q1, Q2 sunt NP-complete atunci Q1 $\leq_p$ Q2 și Q2 $\leq_p$ Q1).

Teorema 4. Dacă există o problemă Q $\in$ NP-completă și Q $\in$ P, atunci P = NP.

Teorema 5. Dacă există o problemă Q$\in$ NP-hard și Q $\in$ P, atunci P = NP.

Demonstrați reducerile polinomiale următoare:

1. 3-Colorare $\leq_p$ 4-Colorare

   Problema **k-Colorare** (k$\geq$2): Dându-se un graf neorientat G = (V, E) și k culori, se pot colora nodurile grafului folosind cele k culori astfel încât niciun nod sa nu aibă un vecin de aceeași culoare?

2. Partitie $\leq_p$ Suma de submulțime / q-Sume

   Problema Partiție: Dându-se un număr n și o mulțime de n întregi A = $\{a_1, ..., a_n\}$, există S $\subset$ {1, ..., n} astfel încât $\sum_{i \in S} a_i = \sum_{i \notin S} a_i$?

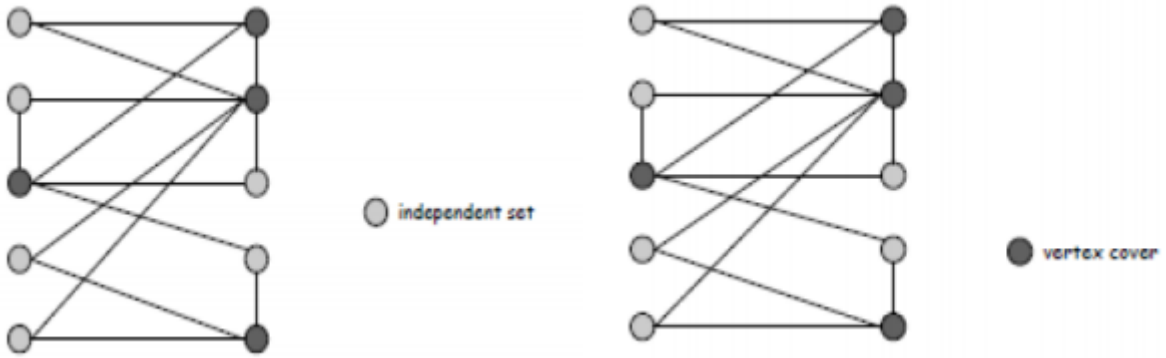
   Problema Suma de submulțime /q-Sume(Subset Sum): Dându-se numerele t, q și o mulțime de t întregi B = $\{b_1, ..., b_t\}$, există $S' \subseteq \{1, ..., t\}$ astfel încât $\sum_{i \in S'} b_i$ = q?

3. Independent Set $\leq_p$ Vertex Cover

   Problema Vertex Cover: Dându-se un graf neorientat G = (V, E) și un număr k, există o submulțime S de noduri, |S| = k, astfel încât fiecare muchie are cel puțin un capăt în S?

   Problema Independent Set: Dându-se un graf neorientat $G' = (V', E')$ și un număr k', există o submulțime S' de noduri, $|S'| = k'$, astfel încât fiecare muchie are cel mult un capăt în S'?

   Exemplu:



4. Vertex Cover $\leq_p$ Set Cover

   Problema Set Cover: Dându-se o mulțime U, o colecție S_1, S_2, ..., S_m de submulțimi ale lui U și un număr k', există o colecție de k' astfel de submulțimi care reunite să dea U?

Corectitudinea Algoritmilor

December 19, 2022

1 Introducere

O analiză utilă a unui algoritm trebuie să aibă în vedere următoarele două aspecte:

- Eficienţa (complexitatea) → cantitatea de resurse (timp şi memorie) solicitate.
- Corectitudinea → pe **orice** date de intrare corecte (i.e. care respectă un set de precondiţii) algoritmul întoarce rezultate corecte (i.e. care respectă un set de postcondiţii).

În general, problema de a demonstra că un algoritm (sau un program) are un anumit efect sau că nu are un anumit efect este nerecursivă sau nici măcar recursiv enumerabilă, după cum am văzut la începutul semestrului. În cazuri speciale, există posibilitatea de a demonstra corectitudinea unor algoritmi, care, bineînțeles, este dezirabilă pentru algoritmii pe care îi dezvoltăm. Importanța unor aplicații (de exemplu prevenirea ciocnirii de trenuri prin semnalizarea automată) și preferința pentru algoritmi demonstrabil corecți dus la dezvoltarea acestui domeniu al cunoașterii.

1.1 Demonstrarea corectitudinii [1]

Pentru a demonstra **corectitudinea**, trebuie să luăm în calcul acest "orice". Demonstraţia corectitudinii se face în **3 paşi**:

1. Identificarea precondiţiilor algoritmului. Totalitatea precondiţiilor formează **aserţiunea de intrare** notată $P_I : I \to \{0, 1\}$, unde $I =$ *domeniul datelor de intrare.*
2. Identificare postcondiţiilor algoritmului. Totalitatea postcondiţiilor formează **aserţiunea de ieşire** notată $P_O : I \times O \to \{0, 1\}$, unde $O =$ *domeniul rezultatelor*
3. Demonstraţia că, plecând din starea iniţială a algoritmului, în care se respectă aserţiunea de intrare, paşii descrişi de algoritm ne duc într-o stare finală în care se respectă aserţiunea de ieşire.

Ideea demonstraţiei este să identificăm forma generală a stărilor intermediare prin care ar trebui să treacă algoritmul şi să ne asigurăm că la fiecare acţiune algoritmul trece dintr-o stare validă în altă stare validă.
Construirea unei asemenea demonstraţii poate ajuta la identificarea şi corectarea erorilor.

1.2 Tipuri de algoritmi corecţi

- Când demonstrăm că un algoritm este *corect* şi că se termină într-un *număr finit de paşi*, spunem că algoritmul este **total corect**:

$$\forall d \in I, P_I(d) \Rightarrow \Big(termin\big(Alg(d)\big) \wedge P_O\big(d, Alg(d)\big) \Big)$$

- Când demonstrăm că un algoritm este *corect* dar doar în cazurile în care se termină într-un *număr finit de paşi*, spunem că algoritmul este **parţial corect**:

$$\forall d \in I, \Big(P_I(d) \wedge termin\big(Alg(d)\big) \Big) \Rightarrow P_O\big(d, Alg(d)\big)$$

2 Invarianţi la ciclare

2.1 Definiţie

Cea mai frecventă tehnică de demonstrare a corectitudinii algoritmilor recursivi sau iterativi este inducţia.
În cazul algoritmilor iterativi, metoda constă în identificarea şi demonstrarea de **invarianţi la ciclare**: proprietăţi valabile la intrarea într-o buclă şi conservate pe durata şi la ieşirea din buclă.

2.2 Demonstrarea invarianţilor

Invarianţii la ciclare se demonstrează în **3 paşi** care corespund, practic, unei inducţii matematice după variabila de ciclare:

1. **Iniţializare**: demonstrăm că invariantul se respectă înainte de prima iterare în buclă (iteraţia 0).
2. **Menţinere**: demonstrăm că dacă invariantul se respectă înainte de iteraţia k, atunci el se va respecta şi dupa iteraţia k (adică înainte de iteraţia k + 1).
3. **Terminare**: invariantul se respectă la ieşirea definitivă din buclă, aşa că precizăm proprietatea utilă rezultată.

2.3 Exemple

2.3.1 Suma elementelor dintr-un vector

```
1: function SUMA(V[0..n − 1])
2:     s = 0
3:     i = 0
4:     while i < n do
5:         s = s + v[i]
6:         i + +
7:     end while
8:     return s
9: end function
```

Invariant: $s_i = \sum_{j=0}^{i-1} v[j]$

1. **Iniţializare:** Înainte de prima iteraţie, $s_0 = 0 = \sum_{j=0}^{-1} v[j]$

2. **Menţinere:** Presupunem că, înainte de iteraţia i, $s_i = \sum_{j=0}^{i-1} v[j]$. După iteraţia i (înainte de iteraţia i + 1) vom avea $s_{i+1} = \sum_{j=0}^{i-1} v[j] + v[i] = \sum_{j=0}^{i} v[j]$.

3. **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $i = n$): $s_n = \sum_{j=0}^{n-1} v[j]$, adică suma tuturor elementelor din vector.

2.3.2 Sortarea prin selecţie

```
1: function SelectionSort(V[0..n − 1])
2:     for i = 0; i < n − 1; i + + do
3:         min = i
4:         for j = i + 1; j < n; j + + do
5:             if v[j] < v[min] then
6:                 min = j
7:             end if
8:         end for
9:         swap(v[i], v[min])
10:    end for
11:    return V
12: end function
```

1. **Invariant pentru bucla interioară:** Înainte de iteraţia j:
   min_j = indexul elementului minim din $v[i..j-1]$

   (a) **Iniţializare:** Înainte de prima iteraţie, $min_{i+1} = i$ = indexul elementului minim din $v[i..i]$

   (b) **Menţinere:** Presupunem că, înainte de iteraţia j,
   min_j = indexul elementului minim din $v[i..j-1]$
   După iteraţia j (înainte de iteraţia j + 1), dacă $v[j] < v[min_j]$, atunci $min_{j+1} = j$, altfel $min_{j+1} = min_j$.
   Aşadar, min_{j+1} = indexul elementului minim din $v[i..j]$

   (c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $j = n$):
   min_n = indexul elementului minim din $v[i..n-1]$

2. **Invariant pentru bucla exterioară:** Înainte de iteraţia i, $v[0..i-1]$ este sortat **şi conţine cele mai mici i elemente din v**.

   (a) **Iniţializare:** Înainte de prima iteraţie, $v[0..-1]$ este sortat şi conţine cele mai mici 0 elemente din v.

   (b) **Menţinere:** Presupunem că, înainte de iteraţia i, $v[0..i-1]$ este sortat şi conţine cele mai mici i elemente din v.
   După iteraţia i (înainte de iteraţia i + 1), $v[0..i-1]$ rămâne nemodificat, iar $v[i]$ primeşte valoarea $v[min]$, unde min = indexul elementului minim din $v[i..n-1]$ (conform invariantului anterior).

Aşadar, înainte de iteraţia i + 1, $v[0..i]$ sortat şi conţine cele mai mici i + 1 elemente din v.

(c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $i = n - 1$): $v[0..n-2]$ este sortat şi conţine cele mai mici n - 1 elemente din v. Rezultă că $v[n-1]$ conţine cel mai mare element din v, deci $v[0..n-1]$ sortat.

Prin invarianții selectați am demonstrat că funcția SelectionSort sortează vectorul V.

3 Exerciţii

3.1 Căutarea binară [2]

```
1: function BinarySearch(V, x, start, stop)
2:     while start ≤ stop do
3:         mid = (start + stop)/2
4:         if (x == v[mid]) then return mid
5:         end if
6:         if (x > v[mid]) then
7:             start = mid + 1
8:         else
9:             stop = mid − 1
10:        end if
11:    end while
12:    return Null
13: end function
```

3.2 Sortarea prin inserţie

```
1: function InsertionSort(V[0..n − 1])
2:     for j = 1; j < n; j + + do
3:         x = v[j]
4:         i = j − 1
5:         while i >= 0 and v[i] > x do
6:             v[i + 1] = v[i]
7:             i − −
8:         end while
9:         v[i + 1] = x
10:    end for
11: end function
```

3.3 Sortarea prin metoda bulelor

```
1: function BubbleSort(V[0..n − 1])
2:     for i = 0; j < n; j + + do
3:         for j = n − 1; j > i; j − − do
4:             if v[j] < v[j − 1] then
5:                 swap(v[j], v[j − 1])
6:             end if
7:         end for
8:     end for
9: end function
```

3.4 Funcţia de partiţionare din cadrul algoritmului QuickSort [3]

```
1: function Partition(V, start, stop)
2:     x = v[stop]
3:     i = start − 1
4:     for j = start; j < stop; + + j do
5:         if v[j] < x then
6:             + + i
7:             swap(v[i], v[j])
8:         end if
9:     end for
10:    swap(v[i + 1], v[stop])
11:    return i + 1
12: end function
```

3.5 Funcţia de interclasare din cadrul algoritmului MergeSort

```
1: function MERGE(A, p, q, r)
2:     n1 = q − p + 1
3:     n2 = r − q
4:     for i = 1; i <= n1; + + i do
5:         L[i] = A[p + i − 1]
6:     end for
7:     L[n1 + 1] = ∞
8:     for i = 1; i <= n2; + + i do
9:         R[i] = A[q + i]
10:    end for
11:    R[n2 + 1] = ∞
12:    i = 1
13:    j = 1
14:    for k = p; k ≤ r; + + k do
15:        if L[i] <= R[j] then
16:            A[k] = L[i]
17:            i = i + 1
18:        else
19:            A[k] = R[j]
20:            j = j + 1
21:        end if
22:    end for
23: end function
```

3.6 Determinarea distanţei minime prin algoritmul lui Dijkstra

```
1: function DIJKSTRA(graph, c, s, adj)
2:     for each v ∈ vertices(graph) do
3:         d[i] = ∞
4:     end for
5:     d[s] = 0
6:     q = priority_queue(graph)
7:     while q do
8:         u = extractMin(q)
9:         q.remove(u)
10:        for each v ∈ adj[u] do
11:            if d[u] + c[u, v] < d[v] then
12:                d[v] = d[u] + c[u, v]
13:            end if
14:        end for
15:    end while
16: end function
```

References

[1] C. A. Giumale, *Introducere în Analiza Algoritmilor*. Polirom, 2004.

[2] "Invariants for binary search." Available at http://www.cs.cornell.edu/courses/cs2112/2015fa/lectures/lec_loopinv/index.html Accessed 10.11.2019.

[3] "Invariants for quicksort." Available at http://www.columbia.edu/~cs2035/courses/csor4231.F03/quicksort-notes.pdf Accessed 10.11.2019.

Inducție Structurală

19 decembrie 2022

Tipuri de date abstracte (TDA)

Un TDA specifică:

- un set de **date (valori care aparțin tipului respectiv)** (ex: O valoare a TDA-ului Stack pentru numere întregi este o stivă care conține elementele 2 și 3, iar toate valorile pentru TDA-ul Stack sunt toate stivele posibile.)
- un set de **operații** care se pot face cu acele date (ex: Pentru TDA-ul Stack operațiile posibile sunt pop și push, prin intermediul cărora generăm noi valori ale TDA-ului.)

De ce "abstract"?

- **independent de implementare** (același TDA poate fi implementat în diverse moduri care respectă definiția)
- vizibil ca model matematic sau programat ca interfață
- abstractizarea structurilor de date prin TDA-uri este importantă pentru că permite programatorilor să folosească structuri de date fără să știe implementarea lor (ex: pop(push(S, x)) va avea același comportament indiferent de limbajul/biblioteca folosită care implementează TDA-ul Stack. Astfel programatorii pot folosi TDA-uri cunoscând doar anumite proprietăți ale operatorilor unui TDA)

Exemplu – Tipul de date abstract Nat

Nume: Nat
Descriere: numere naturale
Tipuri importate: Bool

Constructori de bază:

- $zero :\rightarrow Nat$ # orice număr natural este fie 0
- $succ : Nat \rightarrow Nat$ # fie succesorul altui număr natural

---

Operatori:

- $zero? : Nat \rightarrow Bool$ # testează dacă un număr natural este 0
- $add : Nat * Nat \rightarrow Nat$ # adună 2 numere naturale

---

Axiome:

- $zero?(zero) = true$
- $zero?(succ(n)) = false$
- $add(zero, n) = n$
- $add(succ(m), n) = succ(add(m, n))$

Noțiuni teoretice TDA-uri

Constructori de tip

- toți operatorii unui tip t care **produc o valoare a lui t** se numesc constructori ai acelui tip
- **zero, succ, add** sunt, toți, constructori ai tipului Nat
- zero? nu este constructor pentru Nat, întrucât produce un Bool

Constructori de bază

- un set de constructori **necesari și suficienți** pentru a obține **toate valorile tipului**
- **zero** și **succ** sunt constructori de bază pentru Nat
- add nu este constructor de bază, întrucât nu e necesar; orice număr natural se poate obține din aplicări succesive de zero sau succ

Constructorii unui tip t, din punct de vedere al **parametrilor**

- **nulari**: au 0 argumente; fie Σ_0 mulțimea constructorilor de bază nulari (ex: zero pentru tipul Nat)
- **externi**: au un număr de argumente, dintre care **niciunul nu este de tip t**; fie Σ_e mulțimea constructorilor de bază externi (nu există constructor extern în fișa tipului Nat)
- **interni**: au un număr de argumente, dintre care **cel puțin unul este de tip t**; fie Σ_i mulțimea constructorilor de bază interni (ex: **succ** pentru tipul Nat)

Axiome

- rolul lor este să specifice **cum se comportă toți operatorii TDA-ului pe toate valorile TDA-ului**
- pentru aceasta, e suficient să specifice cum se comportă pe toți **constructorii de bază** (întrucât orice valoare a TDA-ului se poate obține numai din constructorii de bază)
- este extrem de important ca **axiomele să "acopere" toate valorile tipului**
- de asemenea ele trebuie să fie scrise **neredundant** (de exemplu, la axiomele pentru add, am acoperit toate posibilitățile prin a varia doar primul parametru între zero și succ(m), nu era nevoie să procedăm la fel și pentru al doilea; în general, e foarte important (și ușurează mult demonstrațiile) să găsim exprimarea minimală și completă pentru axiome; în particular, când există mai mulți parametri de tip t, considerăm că scriem axiomele din punctul de vedere al unuia dintre ei, nu din al tuturor)

Dimensiunea unei valori v a unui TDA t

- reprezintă numărul aparițiilor constructorilor lui t în formula care desemnează pe v
- ex: succ(succ(zero)) are dimensiunea 3, add(succ(zero), succ(zero)) are dimensiune 4

Inducție structurală

Când este adecvată?

- când algoritmul (sau o parte din el) primește date de un tip t care satisfac o proprietate P (conform aserțiunii de intrare) și trebuie să producă date de tip t care satisfac, de asemenea, proprietatea P (conform aserțiunii de ieșire)
- faptul că proprietatea P este păstrată în urma aplicării algoritmului poate fi demonstrat prin inducție structurală

Moduri de înțelegere a inducției structurale

1. pentru a dovedi că proprietatea P va fi respectată în urma aplicării algoritmului pe toate valorile posibile de tip t, va trebui să arătăm că e respectată de **toți constructorii de bază**

2. pentru a dovedi ca proprietatea P va fi respectată în urma aplicării algoritmului pe toate valorile posibile de tip t, vom face o **inducție după dimensiunea valorilor tipului t** (în cazul de bază vor fi constructorii **nulari** și **externi**, întrucât aceștia sunt singurii de dimensiune 1; **pasul de inducție** se va referi la **constructorii interni**, întrucât aceștia au proprietatea de a incrementa dimensiunea (P(n)→P(n+1), ca la inducția matematica))

Ambele viziuni sunt surprinse de aceeași schemă de inducție structurală:

Cazuri de baza:

$\forall \sigma \in \Sigma_0 \bullet P(\sigma)$

$\forall \sigma \in \Sigma_e, d \in Dom_\sigma \bullet P(\sigma(d))$

Pas de inducție:

$\forall x \in t \bullet P(x) \rightarrow P(\sigma(..., x, ...)), \forall \sigma \in \Sigma_i$

---

atunci $\forall x \in t \bullet P(x)$

Algoritm inducție structurală

Pasul 0: Se alege variabila după care se face inducție.

Pasul 1 (cazul de bază): Corespunde cazului de bază din schema inducției matematice. În cazul inductiei structurale, se arată că proprietatea P este respectată de **toți constructorii nulari și externi**. Acest lucru este necesar deoarece constructorii nulari și externi sunt de dimensiune 1 (nu primesc ca argument tipul **t** deci nu incrementează dimensiunea).

Pasul 2 (ipoteza inductivă): Se presupune că, **pentru fiecare constructor intern**, proprietatea P **este respectată de parametrii de tip t ai fiecărui constructor intern**.

Pasul 3 (pas de inductie): Folosind ipoteza inductivă, se arată că proprietatea P se respectă pe fiecare **constructor intern** (aceștia fiind de dimensiune mai mare ca 1).

Observații generale

- tabela de advăr pentru operatorul → (implicație) arată în felul următor:

$x1$	$x2$	$x1 \rightarrow x2$
False	False	True
False	True	True
True	False	False
True	True	True

- așadar când aveți de demonstrat o implicație discutați cazul când premisa (x1) este falsă, menționând că în acest caz valoarea de adevăr a concluziei (x2) nu mai contează, apoi arătați că atunci când premisa (x1) este adevărată va rezulta automat că concluzia (x2) este adevărată.
- precizați clar axiomele folosite
- demonstrația se face pentru **fiecare constructor în parte** (cazul de bază pentru fiecare constructor nular și extern, pasul de inducție pentru fiecare constructor intern, cu propria ipoteză inductivă)
- inducția se face după **o singură variabilă**
- când nu mai puteți avansa cu demonstrația, aveți de **demonstrat o nouă proprietate prin inducție structurală**. Alegeți cea mai generală proprietate care vă ajută să avansați și **demonstrați-o tot prin inducție structurală, asemenea cerinței**
- trebuie să folosiți ipoteza inductivă la un moment dat
- dacă aveți mai multe valori de adevăr, împărțiți problema în cazuri

Exerciții rezolvate

Exercițiul 1

Fie TDA-ul List descris de fișa următoare:

Nume: List

Tipuri importate: Nat, Elem

Constructori de bază:

- $[] :\rightarrow List$ # orice listă este fie vidă
- $: : Elem * List \rightarrow List$ # fie un element adăugat la o altă listă
  # sintaxa (x:xs) = elem x adăugat la lista xs
  # constructorul este simbolizat prin simbolul ':', celălalt simbol ':' semnifică că urmează definirea operatorului

Operatori:

- $length : List \rightarrow Nat$ # lungimea unei liste
- $++ : List * List \rightarrow List$ # $A++B$ = concatenarea listelor A și B

Axiome:

- $length([]) = 0$ (LEN1)
- $length(x : xs) = 1 + length(xs)$ (LEN2) # shortcut pentru succ(length(xs))
- $[] ++ L = L$ (APP1)
- $(x : xs) ++ L = x : (xs ++ L)$ (APP2)

Cerință: Să se demonstreze, prin inducție structurală, că operația ++ **e asociativă**.
Trebuie demonstrat, așadar, că (A++B)++C = A++(B++C), oricare ar fi A, B, C TDA-uri de tipul List. Vom face inducție structurală dupa A.

Caz de bază:
$A = []$ (constructor nular)
Trebuie arătat că:
$([] ++ B) ++ C = [] ++ (B ++ C) \Leftarrow (APP1) \Rightarrow$
$B ++ C = B ++ C$
(A) oricare ar fi B și C Liste

Pas de inducție:
$A = (x : xs)$ (constructor intern)

Ipoteza inductivă:
$(xs ++ B) ++ C = xs ++ (B ++ C)$ (II)
Trebuie arătat că:
$((x : xs) ++ B) ++ C = (x : xs) ++ (B ++ C) \Leftarrow (APP2) \Rightarrow$
$(x : (xs ++ B)) ++ C = x : (xs ++ (B ++ C)) \Leftarrow (APP2) \Rightarrow$
$x : ((xs ++ B) ++ C) = x : (xs ++ (B ++ C)) \Leftarrow (II) \Rightarrow$
$x : (xs ++ (B ++ C)) = x : (xs ++ (B ++ C))$
(A)
Întrucât am epuizat toți constructorii de bază, rezultă că ++ e asociativ.

Exercitiul 2

Fie TDA-ul SET descris de:

Nume: SET

Tipuri importate: Bool, T

Constructori de bază:

- $[] :\rightarrow SET$ #
- $: : T \times SET \rightarrow SET$ # $a : x$ reprezintă mulțimea formată prin adăugarea elementului a la mulțimea x

Operatori:

- $maj : T \times SET \rightarrow Bool$ # majorant
- $union : SET \times SET \rightarrow SET$ # reuniunea a 2 mulțimi

Axiome:

- $maj(e, []) = true$ (J1)
- $maj(e, a : x) = (e >= a) \wedge maj(e, x)$ (J2)
- $union([], m) = m$ (U1)
- $union(a : x, m) =$ if $member(a, m)$ then $union(x, m)$ else $a : union(x, m)$ (U2)
- $member(e, []) = false$ (M1)
- $member(e, a : x) = (e == a) \vee member(e, x)$ (M2)

Cerință: Să se demonstreze, prin inducție structurală:
$maj(e, union(m1, m2)) \implies maj(e, m1) \wedge maj(e, m2)$.

Vom face inducție structurală după $m1$.

Caz de bază:
$m1 = []$ (constructor nular)
Trebuie aratat că:
$maj(e, union([], m2)) \implies maj(e, []) \wedge maj(e, m2) \Leftarrow (U1, J1) \Rightarrow$
$maj(e, m2) \Rightarrow true \wedge maj(e, m2)$

Pas de inducție:
$m1 = (a : x)$ (constructor intern)

Ipoteza inductivă:
$maj(e, union(x, m2)) \Rightarrow maj(e, x) \wedge maj(e, m2)$ (II)
Demonstram ca:
$maj(e, union(a : x, m2)) \Rightarrow maj(e, a : x) \wedge maj(e, m2)$

1. Pentru **member(a, m2) = false** $\Leftarrow (U2) \Rightarrow$
   $maj(e, a : union(x, m2)) \Rightarrow maj(e, a : x) \wedge maj(e, m2)$

   (a) Pentru $e \geq a \Leftarrow (J2) \Rightarrow$
   $maj(e, union(x, m2)) \Rightarrow maj(e, x) \wedge maj(e, m2) \Leftarrow (II) \Rightarrow$
   $True$.

   (b) Pentru $e < a \Leftarrow (J2) \Rightarrow$
   $False \Rightarrow ceva$.

2. Pentru **member(a, m2) = True** $\Leftarrow (U2, J2) \Rightarrow$
   $maj(e, union(x, m2)) \Rightarrow e \geq a \wedge maj(e, x) \wedge maj(e, m2) \Leftarrow (II, P1 - \text{demonstrata ulterior}) \Rightarrow True.$

In continuare demonstram P1: $member(a, m2) \wedge maj(e, m2) \Rightarrow e \geq a$**.**

Vom face inducție structurala după $m2$.

Caz de bază:
$m2 = []$ (constructor nular)
Trebuie aratat că:
$member(a, []) \wedge maj(e, []) \Rightarrow e \geq a \Leftarrow (M1) \Rightarrow$
$False \Rightarrow ceva$

Pas de inducție:
$m2 = (b : y)$ (constructor intern)

Ipoteza inductivă:
$member(a, y) \wedge maj(e, y) \Rightarrow e \geq a \ (II)$
Demonstram ca:
$member(a, b : y) \wedge maj(e, b : y) \Rightarrow e \geq a \Leftarrow (M2, J2) \Rightarrow$
$(a == b \vee member(a, y)) \wedge (e \geq b) \wedge maj(e, y) \Rightarrow e \geq a$

1. Pentru **a==b**
   $true \wedge e \geq a \wedge maj(e, y) \Rightarrow e \geq a.$

2. Pentru **member(a, y) = True**
   $member(a, y) \wedge e \geq b \wedge maj(e, y) \Rightarrow e \geq a \Leftarrow (II) \Rightarrow True$

Exerciții propuse

1. O expresie aritmetică complet parantezată este definită astfel:
   $0, 1, x, [e1 + e2], [e1 * e2], [-e2]$
   3 constructori nulari (primii 3 enumerați mai sus), 3 constructori interni (ultimii 3 enumerați mai sus).
   Să notăm cu E acest tip de date.
   Se definesc operatorii:
   $\boldsymbol{eval(e, n) : E * N \to N}$

   - (E1) $eval(0, n) = 0$
   - (E2) $eval(1, n) = 1$
   - (E3) $eval(x, n) = n$
   - (E4) $eval([e1 + e2], n) = eval(e1, n) + eval(e2, n)$
   - (E5) $eval([e1 * e2], n) = eval(e1, n) * eval(e2, n)$
   - (E6) $eval([-e1], n) = -eval(e1, n)$

   $\boldsymbol{subst(e, f) : E * E \to E}$

   - (S1) $subst(0, f) = 0$
   - (S2) $subst(1, f) = 1$
   - (S3) $subst(x, f) = f$
   - (S4) $subst([e1 + e2], f) = [subst(e1, f) + subst(e2, f)]$
   - (S5) $subst([e1 * e2], f) = [subst(e1, f) * subst(e2, f)]$
   - (S6) $subst([-e], f) = [-subst(e, f)]$

   Operatorul eval(e, n) substituie în expresia e fiecare apariție a lui x cu n și întoarce rezulatul ei.
   Operatorul subst(e, f) substituie în expresie e fiecare apariție a lui x cu f și întoarce noua expresie formată.
   Să se demonstreze prin inductie structurala ca, pentru orice e, f $\in$ E și n $\in \mathbb{N}$, proprietatea urmatoare este adevarata:
   eval(subst(e, f), n)= eval(e, eval(f, n))

2. Fie tipul de date T LIST, definit prin constructorii de bază:

   - $[] :\to TLIST$
   - $: : T * TLIST \to TLIST$

   Se consideră **operatorii**:
   $\boldsymbol{head(l) : TLIST \backslash [] \to T}$

   - head(a:l) = a

   $\boldsymbol{range(l) : TLIST \to BOOL}$

   - $range([]) = false$
   - $range(a : []) = (a == 1)$
   - $range(a : l) = (a - 1 == head(l))\&\&range(l)$, pentru $l != []$

   $\boldsymbol{sum(l) : TLIST \to \mathbb{N}}$

   - $sum([]) = 0$
   - $sum(a : l) = a + sum(l)$

   Demonstraţi următoarea proprietate prin inducţie structurală:
   $\boldsymbol{range(l) \to sum(l) == head(l) * (head(l) + 1)/2}$

3. Fie tipul de date LIST care reprezintă mulțimea listelor cu numere întregi definit prin următorii constructori de bază:

- $[] :\rightarrow LIST$
- $(x : l) : INT * LIST \rightarrow LIST$

Fie FUN mulțimea funcțiilor definite astfel: $INT * INT \rightarrow INT$.
Se definesc funcțiile (operatori) următoare, cu axiomele aferente:

$\boldsymbol{fold : FUN * INT * LIST \rightarrow INT}$

- $fold(f, z, []) = z$
- $fold(f, z, x : l) = f(x, fold(f, z, l))$

$\boldsymbol{len : LIST \rightarrow INT}$

- $len([]) = 0$
- $len(x : l) = 1 + len(l)$

Să se demonstreze prin inducție structurală că P(l) este adevarată, pentru $\forall$ l $\in$ LIST:
$\boldsymbol{P(l) = (len(l) = fold(inc, 0, l))}$
cu inc definită astfel $inc(x, y) = y + 1$

4. Fie tipul de date asbtract BinTree definit prin constructorii de bază:

   $empty : \rightarrow BinTree$ # Arborele binar vid

   $node : BinTree * Elem * BinTree \rightarrow BinTree$ # Subarborele stâng, radăcina, sub drept

   Se consideră operatorii:

   $\boldsymbol{size : BinTree \rightarrow N}$ # Numărul de elemente din arbore

   $\boldsymbol{all : (Elem \rightarrow Bool) * BinTree \rightarrow Bool}$

   # Întoarce True dacă p(x) este adevărat pentru toate elementele x din Tree, False altfel

   Axiomele pentru All:
   (All1): $all(p, empty) = True$
   (All2): $all(p, node(left, root, right)) = p(root)$ && $all(p, left)$ && $all(p, right)$

   $\boldsymbol{countif : (Elem \rightarrow Bool) * BinTree \rightarrow N}$

   # countif(p, tree) = câte elemente din tree satisfac predicatul p (adică pentru câte elemente din tree p(x) este adevărat

   Scrieți axiomele pentru operatorii size și countif, apoi demonstrați prin inducție structurală:

$$\boldsymbol{all(p, t) \rightarrow countif(p, t) == size(t)} \quad (1)$$